

LAPORAN TUGAS BESAR 2

IF3270 PEMBELAJARAN MESIN

Convolutional Neural Network dan Recurrent Neural Network



Disusun oleh:

Agil Fadillah Sabri	13522006
Bastian H Suryapratama	13522034
Haikal Assyauqi	13522052

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2024/2025

DAFTAR ISI

DAFTAR ISI.....	1
DAFTAR TABEL.....	2
DAFTAR GAMBAR.....	3
BAB I.....	4
DESKRIPSI PERSOALAN.....	4
BAB II.....	5
IMPLEMENTASI.....	5
2.1. Daftar Kelas.....	5
2.2. <i>Forward Propagation</i>	14
BAB III.....	27
PENGUJIAN DAN PEMBAHASAN.....	27
3.1. <i>Convolutional Neural Network (CNN)</i>	27
3.2. <i>Simple Recurrent Neural Network (Simple RNN)</i>	39
3.3. <i>Long Short-Term Memory (LSTM)</i>	47
3.4. Perbandingan Hasil <i>Forward Propagation from Scratch</i> dengan <i>Library</i>	55
BAB IV.....	58
KESIMPULAN DAN SARAN.....	58
4.1. Kesimpulan.....	58
4.2. Saran.....	58
PEMBAGIAN TUGAS.....	59
REFERENSI.....	60

DAFTAR TABEL

Tabel 2.2.1. Contoh Embedding Weights.....	26
Tabel 3.1.1. Hasil Percobaan Pengaruh Jumlah Layer Konvolusi.....	27
Tabel 3.1.2. Hasil Percobaan Pengaruh Banyak Filter per Layer Konvolusi.....	30
Tabel 3.1.3. Hasil Percobaan Pengaruh Ukuran Filter per Layer Konvolusi.....	33
Tabel 3.1.4. Hasil Percobaan Pengaruh Jenis Pooling Layer.....	36
Tabel 3.2.1. Hasil Percobaan Pengaruh Jumlah Layer RNN.....	39
Tabel 3.2.2. Hasil Percobaan Pengaruh Banyak Cell RNN per Layer.....	42
Tabel 3.2.3. Hasil Percobaan Pengaruh Jenis Layer RNN Berdasarkan Arah.....	45
Tabel 3.3.1. Hasil Percobaan Pengaruh Jumlah Layer LSTM.....	47
Tabel 3.3.2. Hasil Percobaan Pengaruh Banyak Cell LSTM per Layer.....	50
Tabel 3.3.3. Hasil Percobaan Pengaruh Jenis Layer LSTM Berdasarkan Arah.....	52
Tabel 3.4.1. Perbandingan Hasil Forward Propagation from Scratch dengan Library untuk CNN.....	55
Tabel 3.4.2. Perbandingan Hasil Forward Propagation from Scratch dengan Library untuk Simple RNN	56
Tabel 3.4.3. Perbandingan Hasil Forward Propagation from Scratch dengan Library untuk LSTM.....	57

DAFTAR GAMBAR

Gambar 2.2.1. Implementasi Tahap Konvolusi dan Detektor.....	14
Gambar 2.2.2. Implementasi Tahap Pooling.....	16
Gambar 2.2.3. Implementasi Global Poling.....	17
Gambar 2.2.4. Implementasi Tahap Flattening.....	18
Gambar 2.2.5. Implementasi Forward Propagation Simple RNN.....	19
Gambar 2.2.6. Ilustrasi return_sequences = True.....	20
Gambar 2.2.7. Ilustrasi return_sequences = False.....	20
Gambar 2.2.8. Implementasi Forward Propagation LSTM.....	21
Gambar 2.2.9. Ilustrasi Cell pada LSTM.....	22
Gambar 2.2.10. Implementasi Forward Propagation Pada Layer Bidirectional.....	23
Gambar 2.2.11. Bidirectional RNN.....	24
Gambar 2.2.12. Pemetaan Input Token ke Vektor Embedding.....	26
Gambar 3.1.1. Grafik Loss Model dengan 1 Layer Konvolusi.....	27
Gambar 3.1.2. Grafik Loss Model dengan 2 Layer Konvolusi.....	28
Gambar 3.1.3. Grafik Loss Model dengan 3 Layer Konvolusi.....	28
Gambar 3.1.4. Grafik Loss Model dengan Konfigurasi Filter [16, 32, 64].....	30
Gambar 3.1.5. Grafik Loss Model dengan Konfigurasi Filter [32, 64, 128].....	31
Gambar 3.1.6. Grafik Loss Model dengan Konfigurasi Filter [64, 128, 256].....	31
Gambar 3.1.7. Grafik Loss Model dengan Ukuran Kernel (3, 3).....	33
Gambar 3.1.8. Grafik Loss Model dengan Ukuran Kernel (5, 5).....	34
Gambar 3.1.9. Grafik Loss Model dengan Ukuran Kernel (7, 7).....	34
Gambar 3.1.10. Grafik Loss Model dengan Max Pooling.....	37
Gambar 3.1.11. Grafik Loss Model dengan Average Pooling.....	37
Gambar 3.2.1. Grafik Loss Model dengan 1 Layer Simple RNN.....	39
Gambar 3.2.2. Grafik Loss Model dengan 2 Layer Simple RNN.....	40
Gambar 3.2.3. Grafik Loss Model dengan 3 Layer Simple RNN.....	40
Gambar 3.2.4. Grafik Loss Model dengan 32 Cell RNN.....	42
Gambar 3.2.5. Grafik Loss Model dengan 64 Cell RNN.....	42
Gambar 3.2.6. Grafik Loss Model dengan 128 Cell RNN.....	43
Gambar 3.2.7. Grafik Loss Model dengan Bidirectional RNN.....	45
Gambar 3.2.8. Grafik Loss Model dengan Unidirectional RNN.....	45
Gambar 3.3.1. Grafik Loss Model dengan 1 Layer LSTM.....	47
Gambar 3.3.2. Grafik Loss Model dengan 2 Layer LSTM.....	48
Gambar 3.3.3. Grafik Loss Model dengan 3 Layer LSTM.....	48
Gambar 3.3.4. Grafik Loss Model dengan 32 Cell LSTM.....	50
Gambar 3.3.5. Grafik Loss Model dengan 64Cell LSTM.....	50
Gambar 3.3.6. Grafik Loss Model dengan 128 Cell LSTM.....	51
Gambar 3.3.7. Grafik Loss Model dengan Bidirectional LSTM.....	53
Gambar 3.3.8. Grafik Loss Model dengan Unidirectional LSTM.....	53

BAB I

DESKRIPSI PERSOALAN

Persoalan ini berfokus pada pengembangan modul *forward propagation from scratch* untuk tiga arsitektur *neural network*: *Convolutional Neural Network* (CNN), *Simple Recurrent Neural Network* (Simple RNN), dan *Long-Short Term Memory* (LSTM). Pengembangan ini dilakukan hanya dengan menggunakan pustaka komputasi numerik dasar, seperti NumPy, untuk proses inti *forward propagation*.

Untuk setiap arsitektur, prosesnya melibatkan pelatihan model awal menggunakan *library* Keras: CNN untuk klasifikasi citra pada *dataset* CIFAR-10, sementara Simple RNN dan LSTM untuk klasifikasi teks pada *dataset* NusaX-Sentiment (Bahasa Indonesia). Selanjutnya, dilakukan analisis terhadap pengaruh berbagai *hyperparameter* (misalnya, jumlah layer, jumlah *filter/unit*, ukuran *filter*, jenis *pooling*, dan arah layer RNN/LSTM) terhadap kinerja model, yang dievaluasi menggunakan metrik *macro f1-score*.

Secara garis besar, implementasi yang perlu dilakukan adalah implementasi modul *forward propagation from scratch* yang mampu membaca bobot dari model Keras yang telah dilatih dan kemudian menghasilkan prediksi. Hasil dari implementasi *from scratch* ini akan diuji dan dibandingkan dengan hasil propagasi maju menggunakan Keras pada data uji. Untuk komponen *dense layer*, diperkenankan menggunakan implementasi *forward propagation Feedforward Neural Network* yang telah ada. Sebagai tambahan, terdapat aspek tambahan (bersifat *optional*) yang mencakup implementasi fungsi *backward propagation from scratch* dan kemampuan menangani *batch inference* pada modul *forward propagation*.

BAB II

IMPLEMENTASI

Implementasi *Forward Propagation* untuk *Convolutional Neural Network* (CNN) dan *Recurrent Neural Network* (RNN) dilakukan menggunakan bahasa pemrograman python (.py) dengan membuat beberapa kelas.

2.1. Daftar Kelas

1. Kelas Layer

Kelas ini berisi implementasi dari beberapa *layer* yang digunakan dalam *forward propagation* dalam *convolutional neural network* (CNN) dan *recurrent neural network* (RNN). Setiap *layer*, setidaknya terdiri atas beberapa *method* public sebagai berikut:

- **get_config(self)**

Sebuah *method* yang mengembalikan konfigurasi *layer* (berupa *dictionary*) yang berisi parameter penting seperti jumlah unit, *activation function*, *kernel size*, dsb. Berguna untuk replikasi atau serialisasi model.

- **set_weights(self, weights)**

Sebuah *method* yang digunakan untuk mengatur bobot *layer* menggunakan daftar atau struktur data bobot yang diberikan. Digunakan untuk memuat bobot hasil pelatihan dari *library* Keras.

- **get_weights(self)**

Sebuah *method* yang mengembalikan bobot *layer* saat ini dalam bentuk *array of weights*.

- **set_input_shape(self, input_shape)**

Sebuah *method* yang digunakan untuk mengatur bentuk (*shape*) *input* yang akan diterima oleh *layer*. Digunakan untuk pengecekan dimensi internal *layer*, seperti ukuran bobot. Parameter *input_shape* yang diterima harus berupa *tuple* yang merepresentasikan dimensi satu buah data (tidak memperhitungkan bentuk *batch*).

- **compute_output_shape(self, input_shape=None)**

Sebuah *method* yang mengembalikan bentuk *output* dari *layer* berdasarkan bentuk *input*. Bermanfaat untuk membangun jaringan secara dinamis.

- **trainable_weights(self)**

Sebuah *method* yang mengembalikan daftar bobot yang dapat dilatih oleh *layer*.

- **__call__(self, inputs)**

Method utama untuk melakukan *forward propagation*. Menerima *input* dan mengembalikan *output* dari *layer*. Biasanya dipanggil ketika *layer* digunakan sebagai fungsi.

berikut adalah daftar kelas *layer* yang diimplementasikan dalam tugas besar dua ini beserta *method* tambahannya selain *method* umum yang telah disebutkan sebelumnya.

a. Kelas Conv2D

- **Daftar Attribute**

- **filters (int)**: Banyaknya filter yang digunakan.
- **kernel_size (tuple)**: Ukuran kernel konvolusi (tinggi, lebar).
- **input_shape (tuple, opsional)**: Bentuk data masukan. Default-nya adalah *None*. Formatnya harus (tinggi, lebar, jumlah *channel*).
- **kernel (np.ndarray, opsional)**: Bobot untuk kernel konvolusi. Default-nya adalah *None*. Kernel harus memiliki bentuk (tinggi_kernel, lebar_kernel, jumlah_channel_input, jumlah_filter).
- **bias (np.ndarray, opsional)**: Bobot untuk bias. Default-nya adalah *None*. Bias harus berbentuk (jumlah_filter,).
- **activation (str, opsional)**: Fungsi aktivasi yang diterapkan setelah konvolusi. Default-nya adalah *None*.
- **strides (tuple, opsional)**: Langkah (*stride*) dari konvolusi. Default-nya adalah (1, 1).
- **padding (str, opsional)**: Jenis *padding*, bisa 'valid' (tanpa *padding*) atau 'same' (Menambahkan *padding* di sekitar *input* agar ukuran *output* sama dengan *input* (jika *stride* = 1)). Default-nya adalah 'valid'.
- ****kwargs**: Argumen tambahan lainnya (*keyword arguments*).

- **Daftar *Method* Tambahan**

- **`__add_padding(self, inputs)`**: *Method* ini bertugas menambahkan *padding* pada data *input* sebelum dilakukan operasi konvolusi, bergantung pada jenis *padding* yang digunakan: "valid" atau "same". Data *input* berupa *array* 3 dimensi (tinggi, lebar, *channel*).
- **`__convolution(self, inputs)`**: *Method* ini melakukan operasi konvolusi 2D secara manual (*from scratch*) terhadap *input*, menggunakan kernel dan *padding* yang telah didefinisikan dalam layer Conv2D. Dipanggil oleh *method* `__call__`.

- b. **Kelas Pooling**

- **Daftar *Attribute***

- **`pool_size (tuple)`**: Ukuran jendela *pooling* (tinggi, lebar).
- **`mode (str)`**: Jenis operasi *pooling*, bisa 'max' (maksimum) atau 'average' (rata-rata). Nilai default adalah 'max'.
- **`input_shape (tuple, optional)`**: Bentuk/ukuran data masukan, harus dalam format (tinggi, lebar, jumlah *channel*). *Default*-nya adalah *None*.
- **`strides (tuple, optional)`**: Langkah pergeseran dari operasi *pooling*. *Default*-nya adalah *None*, yang berarti akan disamakan dengan `pool_size`.
- **`padding (str)`**: Jenis *padding*, bisa 'valid' atau 'same'. *Default*-nya adalah 'valid'.
- **`**kwargs`**: Argumen tambahan lainnya (*keyword arguments*).

- **Daftar *Method* Tambahan**

- **`__add_padding(self, inputs)`**: *Method* ini digunakan untuk menambahkan *padding* pada data *input* berdasarkan tipe *padding* yang dipilih (valid atau same).
- **`__pool(self, inputs)`**: Merupakan *dispatcher* (pengarah) untuk memilih jenis operasi *pooling* yang akan digunakan. Jika *mode*-nya 'max', maka akan memanggil `__max_pool`. Jika *mode*-nya 'average', maka akan memanggil `__average_pool`. Metode ini memastikan bahwa hanya *pooling* tipe 'max' atau 'average' yang valid dijalankan. Dipanggil oleh *method* `__call__`.

- **__max_pool(self, inputs)**: Melakukan *max pooling* terhadap data *input* 4D (*batch_size*, *height*, *width*, *channels*). Setiap jendela *pooling* akan mengambil nilai maksimum dan menuliskannya ke dalam *output*. Dilakukan iterasi untuk setiap *batch*, tinggi, dan lebar dengan langkah (*stride*) tertentu dan menampilkan *progres*.
- **__average_pool(self, inputs)**: Melakukan *average pooling* terhadap data *input* 4D (*batch_size*, *height*, *width*, *channels*). Setiap jendela *pooling* akan dihitung nilai rata-ratanya dan diisi ke dalam *output*. Sama seperti **__max_pool**, dilakukan iterasi per *batch* dan menampilkan *progres*.

Kelas ini memiliki dua anak kelas, yaitu kelas `MaxPooling` dan `AveragePooling`

c. Kelas `GlobalPooling`

- **Daftar *Attribute***
 - **mode (str)**: Jenis operasi *pooling*, bisa 'max' (maksimum) atau 'average' (rata-rata). Nilai default adalah 'max'.
 - **input_shape (tuple, optional)**: Bentuk/ukuran data masukan, harus dalam format (tinggi, lebar, jumlah *channel*). *Default*-nya adalah *None*.
 - ****kwargs**: Argumen tambahan lainnya (*keyword arguments*).
- **Daftar *Method* Tambahan**
 - **__pool(self, inputs)**: *Method* ini menjalankan operasi *pooling* global pada data *input*, baik dalam bentuk *batch* maupun satu sampel. *Pooling* dilakukan dengan cara mereduksi dimensi spasial (tinggi dan lebar) menjadi satu nilai per *channel*. Dipanggil oleh *method* **__call__**.

Kelas ini memiliki dua anak kelas, yaitu kelas `GlobalMaxPooling` dan `GlobalAveragePooling`

d. Kelas Flatten

- **Daftar Attribute**
 - **input_shape (tuple, optional)**: Bentuk/ukuran data masukan, harus dalam format (tinggi, lebar, jumlah *channel*). *Default*-nya adalah *None*.
 - ****kwargs**: Argumen tambahan lainnya (*keyword arguments*).
- **Daftar Method Tambahan**
 - **__flatten(self, inputs)**: Metode ini bertugas untuk mengubah (*reshape*) tensor multidimensi menjadi vektor 1 dimensi, yang dilakukan sebelum memasukkan data ke *fully connected layer*. Dipanggil oleh *method* `__call__`.

e. Kelas Embedding

- **Daftar Attribute**
 - **input_dim (int)**: Ukuran dari kosakata *input* (banyaknya kata berbeda).
 - **output_dim (int)**: Dimensi dari vektor *embedding*.
 - **weights (np.ndarray, opsional)**: Bobot untuk layer *embedding*, harus berbentuk (input_dim, output_dim).
 - **input_shape (tuple, opsional)**: Bentuk dari data input, harus dalam format (panjang_sekuens,).
 - ****kwargs**: Argumen tambahan lainnya (*keyword arguments*).
- **Daftar Method Tambahan**
 - **__embedding_lookup(self, inputs)**: Melakukan pencarian (*lookup*) *embedding* berdasarkan indeks *input*. Dipanggil oleh *method* `__call__`.

f. Kelas SimpleRNN

- **Daftar Attribute**
 - **units (int)**: Jumlah unit keluaran (dimensi dari *hidden state*).
 - **activation (str)**: Fungsi aktivasi yang digunakan. Default adalah "tanh".
 - **kernel (np.ndarray, optional)**: Bobot dari *input* ke *hidden state*, berbentuk (jumlah *input*, *units*).
 - **recurrent_kernel (np.ndarray, optional)**: Bobot dari *hidden state* ke *hidden state*, berbentuk (*units*, *units*).

- **bias (np.ndarray, optional):** Bias untuk *hidden state*, berbentuk (*units*,).
- **input_shape (tuple, optional):** Bentuk data *input*, harus dalam format (*timesteps*, *fitur*), di mana *timesteps* adalah panjang urutan dan fitur adalah ukuran *embedding*.
- **return_sequences (bool):** Menentukan apakah akan mengembalikan hanya *output* terakhir dari *timesteps*, atau seluruh *output timesteps*.
- ****kwargs:** Argumen tambahan lainnya (*keyword arguments*).
- **Daftar Method Tambahan**
 - **__rnn_forward(self, inputs):** Method ini melakukan proses *forward propagation* pada layer SimpleRNN. *Input*-nya berupa tensor dengan bentuk (*batch_size*, *timesteps*, *features*). *Output* yang dihasilkan adalah tensor yang bentuknya tergantung pada parameter *return_sequences*: Jika *return_sequences* bernilai True, output berukuran (*batch_size*, *timesteps*, *units*), yaitu menghasilkan *output* untuk setiap *timestep*. Jika False, *output* berukuran (*batch_size*, *units*), hanya mengembalikan *output* pada *timestep* terakhir. Dipanggil oleh method `__call__`.

g. Kelas LSTM

- **Daftar Attribute**
 - **units (int):** Jumlah unit keluaran (dimensi dari *hidden state*).
 - **activation (str):** Fungsi aktivasi yang digunakan. Default adalah "tanh".
 - **kernel (np.ndarray, optional):** Bobot dari *input* ke *hidden state*, berbentuk (jumlah *input*, $units \times 4$).
 - **recurrent_kernel (np.ndarray, optional):** Bobot dari *hidden state* ke *hidden state*, berbentuk (*units*, $units \times 4$).
 - **bias (np.ndarray, optional):** Bias untuk *hidden state*, berbentuk ($units \times 4$).
 - **input_shape (tuple, optional):** Bentuk data *input*, harus dalam format (*timesteps*, *fitur*).
 - **return_sequences (bool):** Menentukan apakah akan mengembalikan hanya *output* terakhir dari *timesteps*, atau seluruh *output timesteps*.
 - ****kwargs:** Argumen tambahan lainnya (*keyword arguments*).

- **Daftar *Method* Tambahan**

- **`__lstm_forward(self, inputs)`**: Method ini melakukan proses *forward propagation* pada layer LSTM. *Input*-nya berupa tensor dengan bentuk $(batch_size, timesteps, features)$. *Output* yang dihasilkan adalah tensor yang bentuknya tergantung pada parameter `return_sequences`: Jika `return_sequences` bernilai `True`, output berukuran $(batch_size, timesteps, units)$, yaitu menghasilkan *output* untuk setiap *timestep*. Jika `False`, *output* berukuran $(batch_size, units)$, hanya mengembalikan *output* pada *timestep* terakhir. Dipanggil oleh method `__call__`.

h. Kelas **Bidirectional**

- **Daftar *Attribute***

- **`layer (Layer)`**: Layer yang akan dibungkus dalam layer *bidirectional*.
- **`merge_mode (str)`**: Mode untuk menggabungkan keluaran dari layer *forward* dan *backward*. Pilihan yang tersedia adalah 'concat' (digabungkan (ukuran *output* sama dengan ukuran *output* per layer $\times 2$)), 'sum' (penjumlahan elemen *wise* (ukuran *output* sama dengan ukuran *output* per layer)), 'ave' (rata-rata elemen *wise* (ukuran *output* sama dengan ukuran *output* per layer)), dan 'mul' (perkalian elemen *wise* (ukuran *output* sama dengan ukuran *output* per layer)).
- **`weights (list, opsional)`**: Bobot untuk layer *forward* dan *backward*.
- **`Backward_layer (Layer)`**: Layer opsional yang digunakan untuk proses *backward*.
- **`**kwargs`**: Argumen tambahan lainnya (*keyword arguments*).

i. Kelas **Dense**

- **Daftar *Attribute***

- **`units (int)`**: Jumlah unit keluaran (jumlah *hidden neuron*).
- **`input_shape (tuple, opsional)`**: Bentuk data masukan, harus dalam format (unit masukan,).
- **`kernel (np.ndarray, opsional)`**: Bobot dari layer, harus memiliki bentuk (unit masukan, *units*).

- **bias** (**np.ndarray**, **opsional**): Bias dari layer, harus memiliki bentuk (*units*).
- **activation** (**str**, **opsional**): Fungsi aktivasi yang akan diterapkan pada keluaran.
- ****kwargs**: Argumen tambahan lainnya (*keyword arguments*).
- **Daftar Method Tambahan**
 - **__forward(self, inputs)**: Melakukan *forward propagation* pada layer Dense (*fully connected*) untuk menghasilkan *output* dari *input* yang diberikan. Dipanggil oleh *method* `__call__`.

2. Kelas Model

Kelas ini berisi implementasi yang meniru kelas Model Keras. Kelas ini digunakan untuk membangun arsitektur jaringan neural secara modular dan fleksibel, memungkinkan untuk membangun, mengelola, menjalankan prediksi, dan memuat ulang model dari file konfigurasi.

- **Daftar Attribute**
 - **layers** (**list**): Menyimpan daftar *layer* yang akan membentuk arsitektur model. Jika tidak diberikan saat inisialisasi, akan diisi dengan *list* kosong (`[]`).
 - **input_shape** (**tuple**, **opsional**): Menyimpan bentuk *input* (*input shape*) dari data yang masuk ke model. Nilainya dapat diatur kemudian melalui metode `set_input_shape()`.
 - ****kwargs**: Argumen tambahan lainnya (*keyword arguments*).
- **Daftar Method**
 - **add_layer(self, layer)**: Menambahkan sebuah *layer* ke dalam model.
 - **get_layers(self)**: Mengembalikan daftar seluruh *layer* dalam model.
 - **get_layer(self, name=None, index=None)**: Mengambil *layer* berdasarkan nama atau indeks.
 - **set_input_shape(self, input_shape)**: Mengatur bentuk input model dan menyebarkannya ke seluruh *layer* secara berurutan.
 - **get_config(self)**: Mengembalikan konfigurasi model dalam bentuk *dictionary* (*input shape*, konfigurasi tiap *layer*, dan argumen tambahan).

- **predict(self, x):** Melakukan *forward propagation* melalui seluruh *layer* untuk menghasilkan prediksi dari data *input*.
- **summary(self):** Menampilkan ringkasan arsitektur model, termasuk: Nama *layer*, *Output shape*, dan Total parameter.
- **load(self, model_path):** Memuat model dari file yang disimpan (menggunakan `load_model` Keras). Meregenerasi ulang *layer* dan bobot dari file.

3. Kelas ActivationFunction

Kelas ini berisi implementasi untuk menghitung nilai aktivasi dari sebuah nilai. Mendukung 5 fungsi aktivasi, yaitu 'linear', 'relu', 'sigmoid', 'tanh', dan 'softmax'.

- **Daftar Method**

- **__linear(self, x):** Mengembalikan nilai *input* x tanpa perubahan. Fungsi ini adalah fungsi aktivasi linear atau identitas.

$$Linear(x) = x$$

- **__relu(self, x):** Mengembalikan nilai $\max(0, x)$.

$$ReLU(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

- **__sigmoid(self, x):** Mengembalikan hasil fungsi sigmoid.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- **__tanh(self, x):** Mengembalikan hasil dari fungsi *tanh* (*Hyperbolic Tangent*).

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- **__softmax(self, x):** Mengembalikan hasil dari fungsi *softmax*.

$$softmax(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

2.2. Forward Propagation

1. Convolutional Neural Network (CNN)

a. Convolution Stage dan Detector Stage

Tahap konvolusi merupakan tahap untuk mengambil fitur lokal dari *input* (misalnya gambar) dengan menggeser filter (kernel) dan menghitung perkalian elemen *wise* di setiap lokasi. Adapun tahap detector merupakan tahap yang menambahkan non-linearitas ke hasil konvolusi agar model dapat menangkap pola kompleks.

Implementasi *forward propagation* pada tahap konvolusi dan detektor dilakukan oleh kelas Conv2D. Implementasi ini terdapat pada *method* `__convolution`. Berikut adalah implementasinya.

```
1  def __convolution(self, inputs):
2      x = self.__add_padding(inputs)
3
4      # input, kernel, and stride dimensions
5      input_height, input_width, input_channels = x.shape
6      kernel_height, kernel_width, kernel_in_channels, num_filters = self.kernel.shape
7      stride_height, stride_width = self.strides
8
9      # Output shape
10     output_height = (input_height - kernel_height) // stride_height + 1
11     output_width = (input_width - kernel_width) // stride_width + 1
12     output = np.zeros((output_height, output_width, num_filters))
13
14     # Convolution operation
15     for f in range(num_filters):
16         for i in range(output_height):
17             for j in range(output_width):
18                 region = x[
19                     i*stride_height:i*stride_height+kernel_height,
20                     j*stride_width:j*stride_width+kernel_width,
21                     :
22                 ] # shape: (kernel_height, kernel_width, input_channels)
23                 output[i, j, f] = np.sum(region * self.kernel[:, :, :, f])
24     # Add bias if available
25     if self.bias is not None:
26         output[:, :, f] += self.bias[f]
27
28     # Apply activation if available
29     output = self.activation(output)
30
31     return output
```

Gambar 2.2.1. Implementasi Tahap Konvolusi dan Detektor

Proses yang terjadi dalam fungsi di atas yaitu:

1. Penambahan *Padding*

Pada baris 2, *padding* ditambahkan ke *input* jika diperlukan, sesuai pengaturan *layer*.

2. Mengambil Dimensi dari *Input*, *Kernel*, dan *Striding*

Dilakukan pada baris 5-7. *Input* akan memiliki dimensi (tinggi, lebar, banyak *channel*). *Kernel* memiliki dimensi (tinggi, lebar, *channel input*, jumlah filter). *Striding* memiliki dimensi (*horizontal striding*, *vertical striding*) Ini penting untuk menghitung dimensi dari *output* nantinya.

3. Menghitung Ukuran *Output*

Dilakukan pada baris 10-12. Ukuran *output* dihitung berdasarkan ukuran *input* (setelah ditambahkan *padding*), *kernel*, dan *stride*. Setelah ukuran *output* didapatkan, dibuat *array* kosong untuk menampung hasil konvolusi.

4. Tahap Konvolusi

Dilakukan pada baris 15-26. Untuk setiap filter (banyaknya kernel atau *feature map*) yang ada (iterasi terluar), akan dilakukan operasi konvolusi dengan formula sebagai berikut:

$$\text{output neuron} = \text{receptive field} \odot \text{kernel} + \text{bias}$$

Receptive field adalah potongan daerah pada *input* yang akan dilakukan perkalian elemen *wise* dengan kernel. Ini ditunjukkan pada variabel 'region' pada baris 18. *Receptive field* akan bergeser sesuai dengan nilai *stride*. Hasil perkalian ini selanjutnya akan ditambahkan dengan bias dan disimpan pada *output* tensor yang telah diinisialisasi sebelumnya. Simbol ' $\odot$ ' merupakan simbol untuk perkalian elemen *wise*.

5. Tahap *Detector*

Tahap detector dilakukan dengan menerapkan fungsi aktivasi (seperti ReLU, sigmoid, dsb.) ke output konvolusi yang ditunjukkan pada baris 29. Aktivasi ini bertindak sebagai "detektor fitur" dengan memfilter informasi penting.

b. Pooling Stage

Tahap *pooling* adalah tahap dalam CNN yang bertujuan untuk mengurangi dimensi spasial (tinggi dan lebar) dari *feature map* sambil mempertahankan informasi penting. Ini membantu mengurangi beban komputasi, menghindari *overfitting*, dan membuat model lebih tahan terhadap translasi kecil pada input.

Implementasi tahap *pooling* dilakukan oleh kelas Pooling atau GlobalPooling. Implementasi ini terdapat pada *method* `__pool` pada masing masing kelas.

Berikut adalah implementasi dari Pooling (kelas Pooling).

```
1  def __pool(self, inputs):
2      batch_size, height, width, channels = inputs.shape
3      pool_height, pool_width = self.pool_size
4      stride_height, stride_width = self.strides
5
6      output_height = (height - pool_height) // stride_height + 1
7      output_width = (width - pool_width) // stride_width + 1
8
9      output = np.zeros((batch_size, output_height, output_width, channels))
10
11     for b in range(batch_size):
12         for h in range(output_height):
13             h_start = h * stride_height
14             h_end = h_start + pool_height
15             for w in range(output_width):
16                 w_start = w * stride_width
17                 w_end = w_start + pool_width
18                 if self.mode == 'max':
19                     output[b, h, w] = np.max(inputs[b, h_start:h_end, w_start:w_end], axis=(0, 1))
20                 elif self.mode == 'average':
21                     output[b, h, w] = np.mean(inputs[b, h_start:h_end, w_start:w_end], axis=(0, 1))
22
23     return output
```

Gambar 2.2.2. Implementasi Tahap *Pooling*

Proses yang terjadi dalam fungsi di atas yaitu:

1. Mengambil Dimensi dari *Input*, *Pool*, dan *Striding*

Dilakukan pada baris 2-3. *Input* akan memiliki dimensi (tinggi, lebar, banyak *channel*). *Pool* memiliki dimensi (tinggi, lebar). *Striding* memiliki dimensi (*horizontal striding*, *vertical striding*) Ini penting untuk menghitung dimensi dari *output* nantinya.

2. Menghitung Ukuran *Output*

Dilakukan pada baris 6-9. Ukuran *output* dihitung berdasarkan ukuran *input* (setelah ditambahkan *padding*), *pool*, dan *stride*. Setelah ukuran *output* didapatkan, dibuat *array* kosong untuk menampung hasil *pooling*.

3. Tahap *Pooling*

Dilakukan pada baris 11-21. Iterasi dilakukan untuk setiap *sample* data (b) dan *posisi jendela pooling* (h, w). koordinat jendela *pooling* pada *input* disimpan dalam variabel *h_start* dan *h_end* (untuk baris) dan *w_start* dan *w_end* (untuk kolom). Jendela ini akan bergeser sesuai dengan nilai *stride*.

Setelah mendapatkan jendela *pooling*, akan dilakukan proses pooling yang ditunjukkan pada baris 18-21. Jika mode 'max', maka pilih nilai tertinggi dari jendela 2D per *channel*. Jika mode 'average', ambil rata-ratanya.

Pooling dilakukan independen pada tiap *channel*, tidak menggabungkan antar *channel*.

Global pooling adalah versi khusus dari *pooling* dimana seluruh dimensi spasial ($height \times width$) dari setiap *feature map* (*channel*) dipadatkan menjadi satu nilai saja.

Berikut adalah implementasi dari Global Poling (kelas GlobalPooling).

```
1  def __pool(self, inputs):
2      if inputs.ndim == 4:
3          # (batch, height, width, channels) -> (batch, channels)
4          if self.mode == 'max':
5              return np.max(inputs, axis=(1, 2))
6          elif self.mode == 'average':
7              return np.mean(inputs, axis=(1, 2))
8          else:
9              raise ValueError("Invalid mode. Use 'max' or 'average'.")
10     else:
11         # (height, width, channels) -> (channels,)
12         if self.mode == 'max':
13             return np.max(inputs, axis=(0, 1))
14         elif self.mode == 'average':
15             return np.mean(inputs, axis=(0, 1))
16         else:
17             raise ValueError("Invalid mode. Use 'max' or 'average'.")
```

Gambar 2.2.3. Implementasi Global *Poling*

c. *Flattening*

Flattening adalah proses mengubah tensor multi-dimensi menjadi vektor satu dimensi. Pada *Convolutional Neural Networks* (CNN), *flattening* digunakan untuk menghubungkan layer konvolusional dengan layer *fully connected* (*dense*). Misalnya, tensor berukuran (*batch size*, *height*, *width*, *channels*) diubah menjadi (*batch size*, $height \times width \times channels$), tensor berukuran (*height*, *width*, *channels*) diubah menjadi ($height \times width \times channels$).

Tahap *flattening* memiliki tujuan mengubah data spasial menjadi vektor fitur agar bisa diproses oleh layer *dense* (*fully connected*) dan menjembatani antara *feature extraction stage* (CNN) dan *classification stage* (*dense layers*).

Implementasi *forward propagation* pada tahap *flattening* dilakukan oleh kelas *Flattening*. Implementasi ini terdapat pada method `__flatten`. Berikut adalah implementasinya.

```
1  def __flatten(self, inputs):
2      if inputs.ndim == 4:
3          batch_size = inputs.shape[0]
4          return inputs.reshape(batch_size, -1)
5      elif inputs.ndim == 3:
6          return inputs.reshape(-1)
7      else:
8          raise ValueError("Input must be a 3D or 4D tensor.")
```

Gambar 2.2.4. Implementasi Tahap *Flattening*

2. *Recurrent Neural Network* (RNN)

a. *Simple Recurrent Neural Network* (Simple RNN)

Implementasi *forward propagation* untuk *simple* RNN dilakukan oleh kelas *SimpleRNN*. Implementasi ini terdapat pada method `__rnn_forward`. Berikut adalah implementasinya.

```

1  def __rnn_forward(self, inputs):
2      batch_size, timesteps, _features = inputs.shape
3
4      # Initialize hidden state (h_prev)
5      h_prev = np.zeros((batch_size, self.units))
6
7      if self.return_sequences:
8          output_sequences = np.zeros((batch_size, timesteps, self.units))
9
10     for t in range(timesteps):
11         # Get the input at time step t
12         x_t = inputs[:, t, :]
13
14         # SimpleRNN calculation:  $h_t = \text{activation}(x_t @ W_{xh} + h_{t-1} @ W_{hh} + b_h)$ 
15         z = np.dot(x_t, self.kernel) + np.dot(h_prev, self.recurrent_kernel) + self.bias
16         h_t = self.activation(z) # Current hidden state
17
18         if self.return_sequences:
19             output_sequences[:, t, :] = h_t
20
21         h_prev = h_t # Update hidden state for the rnn timestep
22
23     if self.return_sequences:
24         return output_sequences
25     else:
26         return h_t # Return the last hidden state if return_sequences is False

```

Gambar 2.2.5. Implementasi *Forward Propagation Simple RNN*

Proses yang terjadi dalam fungsi di atas yaitu:

1. Inisialisasi *Hidden State*

Pada baris 5-8, *hidden state* diinisialisasi dengan nilai 0 sesuai dengan jumlah unit di layer RNN (ukuran *hidden state*).

2. Iterasi per *Timestep*

Pada baris 10-16, untuk setiap *timestep*, ambil input x_t pada timestep ke- t . Kemudian lakukan perhitungan nilai *hidden state timestep* ke- t dengan formula sebagai berikut:

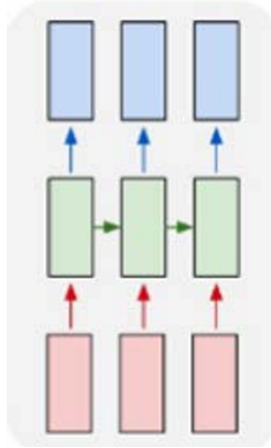
$$h_t = \tanh(x_t U + h_{t-1} W + b_{xh})$$

Dalam fungsi di atas, U merupakan `self.kernel`, W merupakan `self.recurrent_kernel`, b_{xh} merupakan `self.bias`, x_t merupakan `x_t`, dan h_{t-1} merupakan `h_prev`.

- $x_t U$: interaksi *input* dan bobot *input*.
- $h_{(t-1)} W$: interaksi *hidden state* sebelumnya dan bobot rekuren.
- b_{xh} : bias dari input ke unit RNN.

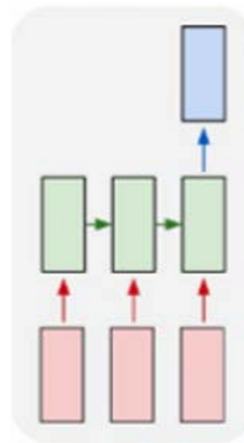
3. Menyimpan dan Mengembalikan *Output*

Jika `return_sequences = True`, simpan dan kembalikan semua *hidden state* dari setiap *timesteps*. Ilustrasi seperti gambar di bawah.



Gambar 2.2.6. Ilustrasi `return_sequences = True`

Jika `return_sequences = False`, hanya *hidden state* dari *timestep* terakhir yang akan disimpan dan dikembalikan. Ilustrasi seperti gambar di bawah.



Gambar 2.2.7. Ilustrasi `return_sequences = False`

b. *Long Short-Term Memory* (LSTM)

Implementasi *forward propagation* untuk LSTM dilakukan oleh kelas LSTM. Implementasi ini terdapat pada method `__lstm_forward`. Berikut adalah implementasinya.

```

1  def __lstm_forward(self, inputs):
2      batch_size, timesteps, features = inputs.shape
3      output = np.zeros((batch_size, timesteps, self.units)) if self.return_sequences
4                      else np.zeros((batch_size, self.units))
5
6      h_t = np.zeros((batch_size, self.units)) # Hidden state
7      c_t = np.zeros((batch_size, self.units)) # Cell state
8
9      for t in range(timesteps):
10         # Get the input at time step t
11         x_t = inputs[:, t, :]
12         z = np.dot(x_t, self.kernel) + np.dot(h_t, self.recurrent_kernel) + self.bias
13
14         # z will have shape = [input_gate | forget_gate | cell_gate | output_gate]
15         # Split z into input gate, forget gate, cell gate, and output gate
16         # Input Gate: first self.units elements
17         i_t = self.recurrent_activation(z[:, :self.units])
18         # Forget Gate: second self.units elements
19         f_t = self.recurrent_activation(z[:, self.units:self.units * 2])
20         # Cell Gate: third self.units elements
21         c_tilde_t = self.activation(z[:, self.units * 2:self.units * 3])
22         # Output Gate: fourth self.units elements
23         o_t = self.recurrent_activation(z[:, self.units * 3:])
24
25         c_t = f_t * c_t + i_t * c_tilde_t
26         h_t = o_t * self.activation(c_t)
27
28         if self.return_sequences:
29             output[:, t, :] = h_t
30
31     if not self.return_sequences:
32         output = h_t
33
34     return output

```

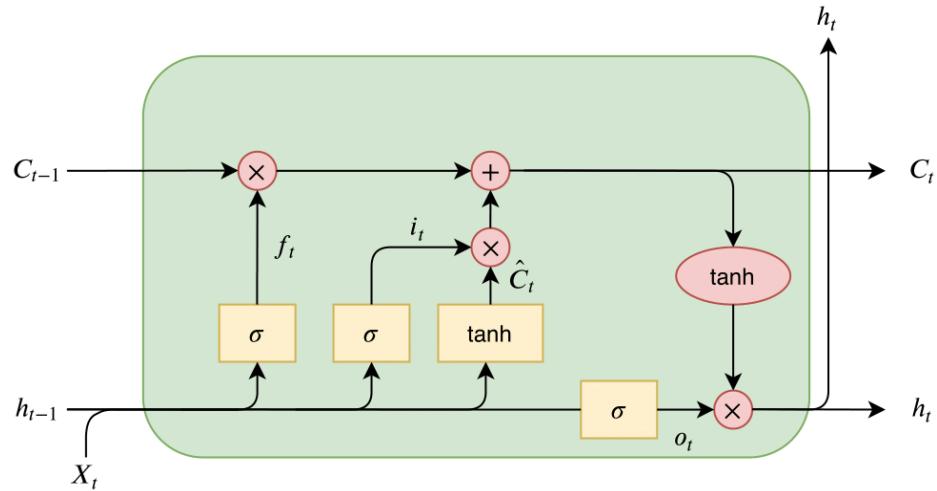
Gambar 2.2.8. Implementasi *Forward Propagation* LSTM

Proses yang terjadi dalam fungsi di atas yaitu:

1. Inisialisasi *Hidden State* dan *Cell State*

Dilakukan pada baris 6-7. *Hidden state* (h_t) digunakan untuk *output* tiap *timesteps*. Sedangkan *cell state* (c_t) menyimpan memori jangka panjang.

2. Iterasi per *Timestep*



Gambar 2.2.9. Ilustrasi *Cell* pada LSTM

Sumber: <https://github.com/ananyasgit/Spam-ham-Prediction-using-LSTM>

Pada baris 9-29, untuk setiap *timestep*, ambil input x_t pada *timestep* ke- t . Kemudian lakukan perhitungan nilai *hidden state* dan *cell state timestep* ke- t .

Dikarenakan implementasi dari Keras menggabungkan setiap bobot untuk tiap *gate* ke dalam satu *list* saja, maka proses perhitungan dapat dilakukan dalam satu kali operasi saja, seperti yang ditunjukkan pada baris 12. *List* bobot tersebut akan memiliki urutan [*input gate* | *forget gate* | *cell gate* | *output gate*].

Untuk memudahkan perhitungan, nilai masing-masing *gate* kemudian dipisah, seperti yang dilakukan pada baris 14-23, dengan rincian:

- *Input Gate* (i_t): Mengatur seberapa banyak informasi baru masuk ke sel.
- *Forget Gate* (f_t): Mengontrol informasi lama yang dibuang dari sel.
- *Cell Candidate* ($\hat{c}_t$): Calon nilai baru untuk *cell state*.
- *Output Gate* (o_t): Menentukan berapa banyak *cell state* yang dikirim ke *output*.

Selanjutnya, dilakukan update *cell state* dan *hidden state*, dengan formula sebagai berikut (dilakukan pada baris 25-26):

$$c_t = f_t \times c_{t-1} + i_t \times \hat{c}_t$$

$$h_t = o_t \times \tanh(c_t)$$

3. Menyimpan dan Mengembalikan *Output*

Jika `return_sequences = True`, simpan dan kembalikan semua *hidden state* dari setiap *timesteps*. Jika `return_sequences = False`, hanya *hidden state* dari *timestep* terakhir yang akan disimpan dan dikembalikan.

c. *Bidirectional Layer*

Bidirectional layer memungkinkan sebuah lapisan RNN (seperti LSTM atau SimpleRNN) untuk membaca urutan *input* dalam dua arah, maju (*forward*) dan mundur (*backward*). *Bidirectional layer* digunakan dalam konteks pemrosesan sekuensial (misalnya NLP) untuk menangkap informasi masa lalu dan masa depan dalam sekuens serta mengatasi keterbatasan RNN satu arah yang hanya melihat konteks sebelumnya.

Implementasi sebelumnya hanya mendukung *unidirectional* RNN. Agar bisa melakukan *bidirectional* RNN/LSTM dibuat kelas *Bidirectional*. Adapun *forward propagation* untuk kelas *Bidirectional* diimplementasikan pada *method* `__call__`. Berikut adalah implementasinya.

```
1  def __call__(self, inputs):
2      forward_output = self.layer(inputs)
3      backward_output = self.backward_layer(np.flip(inputs, axis=1))
4
5      if isinstance(backward_output, np.ndarray) and backward_output.ndim == 3:
6          backward_output = np.flip(backward_output, axis=1)
7
8
9      if self.merge_mode == 'concat':
10         return np.concatenate([forward_output, backward_output], axis=-1)
11     elif self.merge_mode == 'sum':
12         return forward_output + backward_output
13     elif self.merge_mode == 'ave':
14         return (forward_output + backward_output) / 2
15     elif self.merge_mode == 'mul':
16         return forward_output * backward_output
17     else:
18         raise ValueError("Unsupported merge mode: {}".format(self.merge_mode))
```

Gambar 2.2.10. Implementasi *Forward Propagation* Pada *Layer Bidirectional*

Proses yang terjadi dalam fungsi di atas yaitu:

1. *Forward Pass* (Arah Maju)

Proses *input sekuens* dari awal ke akhir (*left to right*). Atribut `self.layer` adalah *instance* dari layer RNN (misalnya SimpleRNN atau LSTM). Proses ini menghasilkan *forward output* dengan dimensi (*batch_size, timesteps, units*) atau (*batch_size, units*) tergantung layer.

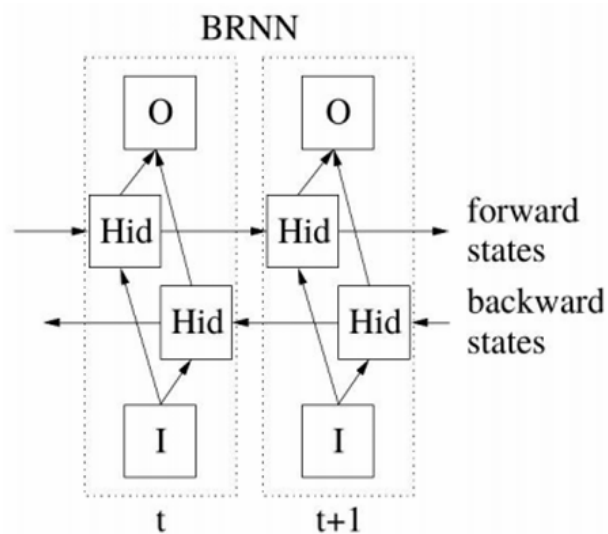
2. *Backward Pass* (Arah Mundur)

Input dibalik urutannya dengan `np.flip(..., axis=1)`, yaitu membalik sepanjang dimensi *timesteps*. Diproses oleh `self.backward_layer`, yaitu *instance* lain dari *layer* yang identik tapi membaca mundur.

3. *Kembalikan Urutan Backward*

Jika *output backward* memiliki dimensi 3 (saat `return_sequences=True`), hasil dibalik lagi ke urutan semula agar sinkron dengan *output forward*. Hal ini perlu dilakukan karena *input* sebelumnya dibalik, sehingga *output* yang dihasilkan pun terbalik (dari *timesteps* terakhir ke pertama) sehingga tidak sinkron dengan *output forward* (dari *timesteps* pertama ke terakhir). Hal ini penting agar posisi *timestep* cocok saat penggabungan.

4. *Merge Output Forward dan Backward*



Gambar 2.2.11. *Bidirectional RNN*

Penggabungan *output* dilakukan dengan salah satu strategi *merge*:

- *Concat*

Cara penggabungan: Gabungkan secara horizontal (*feature axis*).

Dimensi *output*: (*batch, timesteps, units * 2*)

Contoh: *Forward output*: [1, 2, 3]

Backward output: [5, 6, 7]

 Hasil:[1, 2, 3, 5, 6, 7]

- *Sum*

Cara penggabungan: Jumlahkan elemen-elemen *output*.

Dimensi *output*: (*batch, timesteps, units*)

Contoh: *Forward output*: [1, 2, 3]

Backward output: [5, 6, 7]

 Hasil:[6, 8, 10]

- *Average*

Cara penggabungan: Rata-rata elemen-elemen *output*.

Dimensi *output*: (*batch, timesteps, units*)

Contoh: *Forward output*: [1, 2, 3]

Backward output: [5, 6, 7]

 Hasil:[3, 4, 5]

- *Multiplication*

Cara penggabungan: Perkalian elemen-elemen *output*.

Dimensi *output*: (*batch, timesteps, units*)

Contoh: *Forward output*: [1, 2, 3]

Backward output: [5, 6, 7]

 Hasil:[5, 12, 21]

d. *Embedding*

Embedding adalah representasi vektor berdimensi rendah dari item berdimensi tinggi atau diskrit (seperti kata atau ID token) yang memungkinkan sistem pembelajaran mesin memahami hubungan semantik antar item.

Dalam implementasi ini, *embedding* akan memetakan *input* token (representasi *integer* non negatif dari tiap kata unik dalam suatu bahasa) menjadi representasi kontinu (*float vector*) untuk selanjutnya diproses oleh *layer* RNN. Hal ini berguna untuk menangkap hubungan semantik antar token (misal, "Paris" dan "France" akan lebih dekat dalam ruang vektor).

Implementasi *forward propagation* untuk *embedding* dilakukan oleh kelas *Embedding* pada method `__embedding_lookup`. Berikut adalah implementasinya.

```
1 def __embedding_lookup(self, inputs):
2     return self.weights[inputs]
```

Gambar 2.2.12. Pemetaan *Input* Token ke Vektor *Embedding*

Proses yang terjadi dalam fungsi di atas yaitu:

1. Bobot *Embedding*

Bobot disimpan dalam `self.weights`, yang merupakan sebuah matriks dengan dimensi (`vocab_size`, `output_dim`), di mana:

- `vocab_size` = jumlah kata/ID unik dalam kosakata.
- `output_dim` = dimensi vektor *embedding* untuk tiap kata.

2. Cara Kerja

Misalkan terdapat *weights* sebagai berikut:

Tabel 2.2.1. Contoh *Embedding Weights*

Token	<i>Embedding</i>	Token	<i>Embedding</i>
1	[0.3, 0.4]	3	[0.7, 0.8]
2	[0.5, 0.6]	4	[0.9, 1.0]

Maka token `[[2, 4], [3, 1]]` akan menghasilkan vektor:

`[[[0.5, 0.6], [0.9, 1.0]], [[0.7, 0.8], [0.3, 0.4]]]`

BAB III

PENGUJIAN DAN PEMBAHASAN

3.1. *Convolutional Neural Network (CNN)*

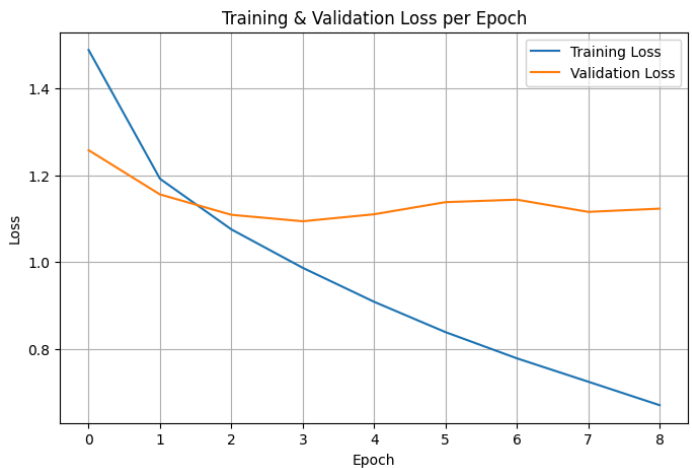
1. Pengaruh Jumlah *Layer* Konvolusi

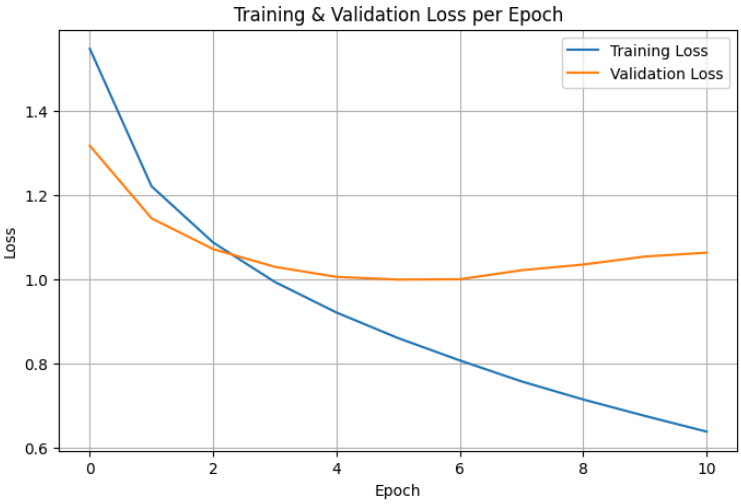
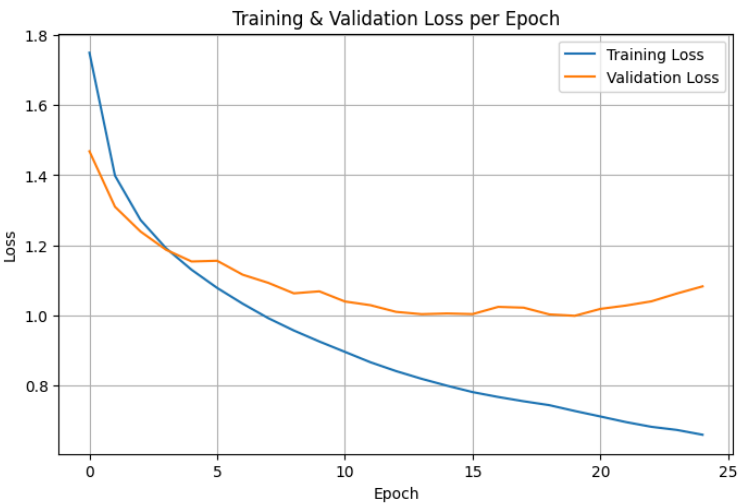
Untuk melihat bagaimana pengaruh jumlah *layer* konvolusi terhadap model, *hyperparameter* berikut dibuat tetap:

- Jumlah filter per *layer* konvolusi: 32,
- Ukuran filter per *layer* konvolusi: (3, 3),
- Fungsi aktivasi pada *layer* konvolusi: ReLU,
- *Pooling layer*: *Max Pooling* dengan ukuran *pool* (2, 2),
- *Batch size*: 64,
- *Epoch*: 30,
- Satu *flatten layer*,
- Satu *dense layer* dengan 128 neuron, fungsi aktivasi ReLU,
- Satu *dense layer* dengan 10 neuron sebagai *output layer*, fungsi aktivasi *softmax*.

a. Hasil Pengujian

Tabel 3.1.1. Hasil Percobaan Pengaruh Jumlah *Layer* Konvolusi

Jumlah <i>Layer</i> Konvolusi	Macro F1-Score	Grafik Training Loss dan Validation Loss
1 <i>Layer</i>	0.6228	 Gambar 3.1.1. Grafik <i>Loss</i> Model dengan 1 <i>Layer</i> Konvolusi

2 Layer	0.6539	 Gambar 3.1.2. Grafik <i>Loss</i> Model dengan 2 Layer Konvolusi
3 Layer	0.6736	 Gambar 3.1.3. Grafik <i>Loss</i> Model dengan 3 Layer Konvolusi

b. Pembahasan

- Hasil Akhir Prediksi (*Macro F1-Score*)

Berdasarkan hasil pengujian yang dilakukan, terlihat bahwa penambahan jumlah *layer* konvolusi memberikan peningkatan konsisten pada *macro F1-score*, menandakan model semakin mampu mengekstrak fitur yang lebih kompleks dari data, yang mengarah pada performa klasifikasi yang lebih baik untuk dataset ini. Peningkatan terbesar terjadi saat beralih dari 1 ke 2 layer.

- Perbandingan Grafik *Training & Validation Loss*

1. Satu *Layer* Konvolusi

Training Loss menurun secara stabil selama 8 *epoch*. *Validation loss* turun dari dan mencapai titik terendah pada *epoch* ke-3, kemudian cenderung stagnan dan sedikit naik pada *epoch* ke-8. Model ini belajar dengan cepat, namun *overfitting* mulai terlihat setelah *epoch* ke-3, di mana *validation loss* berhenti menurun signifikan dan mulai sedikit meningkat. Celah antara *training* dan *validation loss* cukup jelas.

2. Dua *Layer* Konvolusi

Training Loss menurun secara stabil selama 10 *epoch*. Penurunan *training loss* lebih baik dibandingkan model 1 layer. *Validation loss* turun dari sekitar dan mencapai titik terendah pada *epoch* ke-5 dan ke-6, kemudian mulai meningkat pada *epoch* ke-10. Model ini mampu mencapai *validation loss* yang lebih rendah dibandingkan model 1 layer. *Overfitting* mulai terlihat sekitar *epoch* ke-6. Celah antara *training* dan *validation loss* juga jelas dan cenderung melebar.

3. Tiga *Layer* Konvolusi

Training Loss menurun lebih stabil selama 24 *epoch*. Model ini dilatih untuk lebih banyak *epoch* dan mencapai *training loss* yang rendah. *Validation loss* turun dan mencapai titik terendah yang bertahan cukup lama (dari sekitar *epoch* ke-13 hingga ke-17), kemudian mulai meningkat hingga sekitar 1.08 pada *epoch* ke-24. Model ini mencapai *validation loss* minimum yang serupa dengan model 2 layer, namun mampu mempertahankannya lebih lama sebelum *overfitting* menjadi lebih jelas. *Overfitting* mulai terlihat dominan setelah *epoch* ke-17. Celah antara *training* dan *validation loss* menjadi sangat lebar, menunjukkan *overfitting* yang signifikan pada akhir *training*.

- Kesimpulan Perbandingan Grafik *Training & Validation Loss*

Menambah jumlah *layer* konvolusi secara umum meningkatkan kemampuan model dalam mengekstrak fitur, yang berdampak positif terhadap F1-score dan stabilitas *loss*.

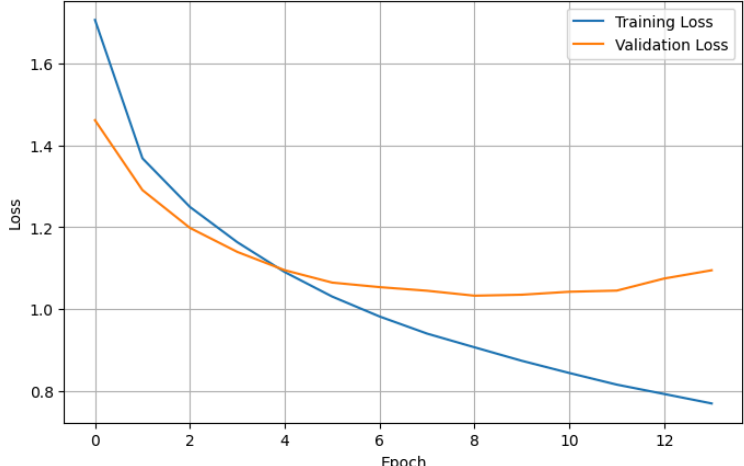
2. Pengaruh Banyak Filter per *Layer* Konvolusi

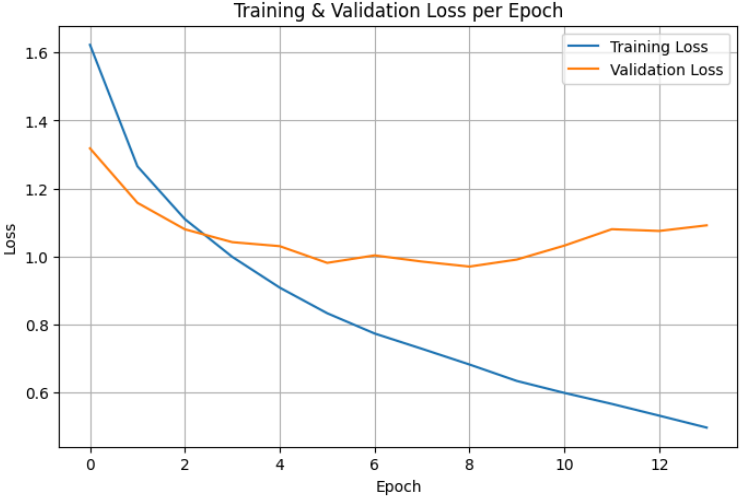
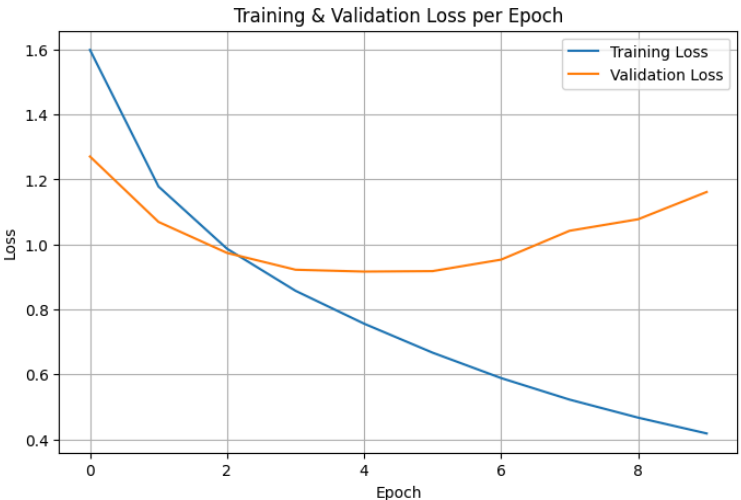
Untuk melihat bagaimana pengaruh banyaknya filter per *layer* konvolusi terhadap model, *hyperparameter* berikut dibuat tetap:

- Jumlah *layer* konvolusi: 3 *layer*,
- Ukuran filter per *layer* konvolusi: (3, 3),
- Fungsi aktivasi pada *layer* konvolusi: ReLU,
- *Pooling layer*: Max *Pooling* dengan ukuran *pool* (2, 2),
- *Batch size*: 64,
- *Epoch*: 30,
- Satu *flatten layer*,
- Satu *dense layer* dengan 128 neuron, fungsi aktivasi ReLU,
- Satu *dense layer* dengan 10 neuron sebagai *output layer*, fungsi aktivasi *softmax*.

a. Hasil Pengujian

Tabel 3.1.2. Hasil Percobaan Pengaruh Banyak Filter per *Layer* Konvolusi

Jumlah Filter tiap <i>Layer</i>	Macro F1-Score	Grafik Training Loss dan Validation Loss
[16, 32, 64]	0.6416	 Gambar 3.1.4. Grafik <i>Loss</i> Model dengan Konfigurasi Filter [16, 32, 64]

[32, 64, 128]	0.6908	 Gambar 3.1.5. Grafik <i>Loss</i> Model dengan Konfigurasi Filter [32, 64, 128]
[64, 128, 256]	0.6942	 Gambar 3.1.6. Grafik <i>Loss</i> Model dengan Konfigurasi Filter [64, 128, 256]

b. Pembahasan

- Hasil Akhir Prediksi (*Macro F1-Score*)

Berdasarkan hasil pengujian yang dilakukan, terlihat bahwa meningkatkan jumlah filter per *layer* memperkuat kapasitas model untuk menangkap detail fitur yang lebih kaya. Namun, terlalu banyak filter juga dapat menyebabkan model terlalu kompleks. Penambahan filter lebih lanjut mungkin tidak memberikan peningkatan performa yang sepadan dan bisa meningkatkan risiko *overfitting* serta kebutuhan komputasi.

- Perbandingan Grafik *Training & Validation Loss*

1. 16, 32, dan 64 Filter

Training loss menurun secara konsisten selama 13 *epoch*. *Validation loss* turun dan mencapai titik terendah *epoch* ke-8 atau ke-9, kemudian mulai sedikit naik pada *epoch* ke-13. Model ini belajar, namun sedikit *overfitting* di mana *validation loss* mulai meningkat sementara *training loss* terus menurun. Celah antara keduanya cukup kecil di akhir.

2. 32, 64, dan 128 Filter

Training loss menurun secara konsisten selama 13 *epoch*. Penurunan lebih tajam dan mencapai nilai lebih rendah dibandingkan konfigurasi filter sebelumnya. *Validation loss* turun dan mencapai titik terendah pada *epoch* ke-8, kemudian meningkat lebih jelas pada *epoch* ke-13. Model ini belajar lebih baik pada data *training* (*training loss* lebih rendah). Namun, *overfitting* juga lebih jelas terlihat. Celah antara *training* dan *validation loss* di akhir lebih lebar dibandingkan konfigurasi [16, 32, 64], menandakan *overfitting* yang lebih kuat.

3. 64, 128, dan 256 Filter

Training loss menurun sangat tajam dan cepat selama 9 *epoch*. Ini adalah *training loss* terendah di antara ketiganya. *Validation loss* turun dari dan mencapai titik terendah *epoch* ke-3 atau ke-4, kemudian meningkat secara signifikan dan cepat pada *epoch* ke-9. Model ini memiliki kapasitas paling besar dan belajar sangat cepat pada data *training*. Namun, ini juga menunjukkan *overfitting* yang paling parah dan terjadi paling awal (sekitar *epoch* ke-4). Celah antara *training* dan *validation loss* sangat lebar dan *validation loss* meningkat paling tajam, menunjukkan kesulitan generalisasi yang signifikan.

- Kesimpulan Perbandingan Grafik *Training & Validation Loss*

Penambahan filter secara umum meningkatkan kemampuan model dalam mengekstrak fitur, namun perlu dibatasi agar tidak *overfit*. Kombinasi [32, 64, 128] memberikan hasil paling seimbang antara performa dan stabilitas.

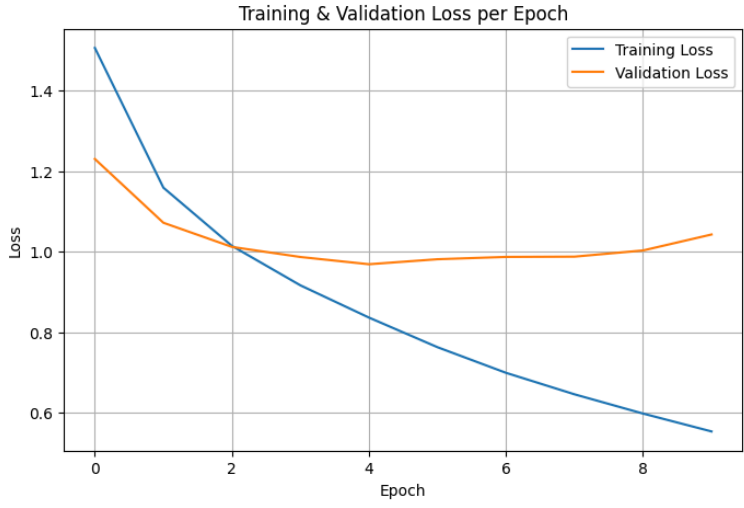
3. Pengaruh Ukuran Filter per *Layer* Konvolusi

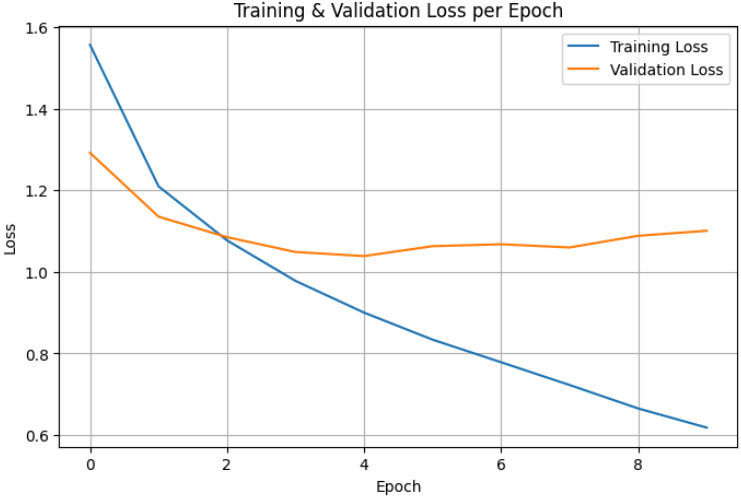
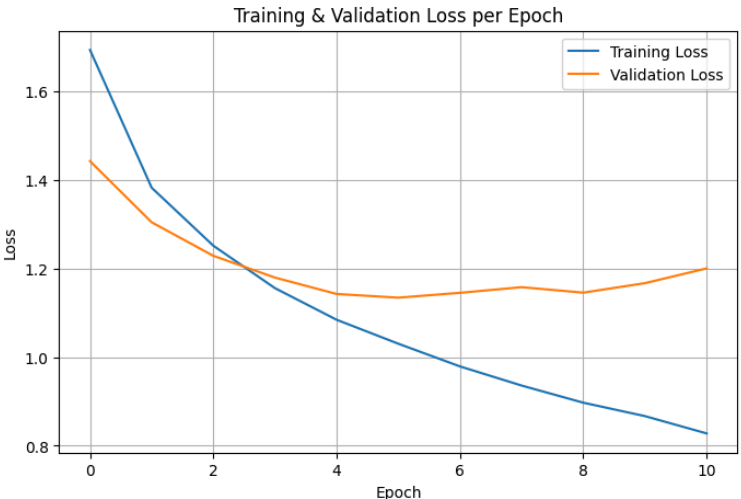
Untuk melihat bagaimana pengaruh ukuran filter per *layer* konvolusi terhadap model, *hyperparameter* berikut dibuat tetap:

- Jumlah *layer* konvolusi: 2 *layer* dengan 32 filter dan 64 filter,
- Fungsi aktivasi pada *layer* konvolusi: ReLU,
- *Pooling layer*: *Max Pooling* dengan ukuran *pool* (2, 2),
- *Batch size*: 64,
- *Epoch*: 30,
- Satu *flatten layer*,
- Satu *dense layer* dengan 128 neuron, fungsi aktivasi ReLU,
- Satu *dense layer* dengan 10 neuron sebagai *output layer*, fungsi aktivasi *softmax*.

a. Hasil Pengujian

Tabel 3.1.3. Hasil Percobaan Pengaruh Ukuran Filter per *Layer* Konvolusi

Ukuran Filter tiap <i>Layer</i>	<i>Macro F1-Score</i>	Grafik Training Loss dan Validation Loss
(3, 3)	0.6692	 Gambar 3.1.7. Grafik <i>Loss</i> Model dengan Ukuran Kernel (3, 3)

(5, 5)	0.6470	 Gambar 3.1.8. Grafik <i>Loss</i> Model dengan Ukuran Kernel (5, 5)
(7, 7)	0.5980	 Gambar 3.1.9. Grafik <i>Loss</i> Model dengan Ukuran Kernel (7, 7)

b. Pembahasan

- Hasil Akhir Prediksi (*Macro F1-Score*)

Berdasarkan hasil pengujian, penggunaan ukuran filter yang lebih kecil menghasilkan *macro F1-score* yang lebih tinggi. Terlihat adanya penurunan performa (*macro F1-score*) seiring dengan meningkatnya ukuran filter pada layer konvolusi. Hal ini mengindikasikan bahwa untuk dataset dan arsitektur yang diuji, filter dengan ukuran (3,3) lebih efektif dalam mengekstraksi fitur-fitur yang relevan untuk klasifikasi dibandingkan filter yang lebih besar. Filter yang lebih kecil cenderung menangkap detail lokal dan pola yang lebih halus, yang mungkin

lebih penting untuk tugas ini. Sebaliknya, filter yang lebih besar menangkap fitur yang lebih global dan mencakup area yang lebih luas, yang mungkin menyebabkan hilangnya informasi detail atau penggabungan fitur yang kurang relevan.

- Perbandingan Grafik *Training & Validation Loss*

1. Ukuran Filter 3×3

Training loss menurun secara konsisten selama 9 *epoch*. *Validation loss*, menurun hingga mencapai titik terendah pada *epoch* ke-4, kemudian cenderung stagnan dan sedikit meningkat hingga *epoch* ke-9. Model ini menunjukkan kemampuan belajar yang baik pada data *training*. Terdapat indikasi *overfitting* yang mulai terlihat setelah *epoch* ke-4, di mana *validation loss* berhenti menurun dan mulai sedikit naik, sementara *training loss* terus menurun. Namun, celah antara *training* dan *validation loss* masih relatif terkendali dibandingkan variasi lain pada titik akhirnya.

2. Ukuran Filter 5×5

Training loss menurun secara konsisten selama 9 *epoch*. *Validation loss* menurun hingga mencapai titik terendah pada *epoch* ke-4, kemudian cenderung stagnan dan sedikit meningkat pada *epoch* ke-9. Pola yang mirip dengan filter (3,3), namun *validation loss* minimumnya sedikit lebih tinggi. Indikasi *overfitting* juga mulai terlihat sekitar *epoch* ke-4. Celah antara *training* dan *validation loss* di akhir tampak sedikit lebih lebar dibandingkan dengan filter (3,3).

3. Ukuran Filter 7×7

Training loss menurun lebih lambat dibandingkan sebelumnya selama 10 *epoch*. Penurunan *training loss* tidak serendah pada filter yang lebih kecil. *Validation loss* menurun hingga mencapai titik terendah *epoch* ke-5, kemudian cenderung meningkat lebih jelas pada *epoch* ke-10. Model ini menunjukkan *training loss* yang lebih tinggi dibandingkan dua model lainnya pada *epoch* terakhir yang ditampilkan. *Overfitting* juga tampak lebih jelas, dengan peningkatan *validation loss* yang lebih signifikan setelah mencapai titik

minimumnya. Celah antara *training* dan *validation loss* adalah yang paling lebar di antara ketiga variasi.

- Kesimpulan Perbandingan Grafik *Training & Validation Loss*,

Ukuran filter kecil seperti (3, 3) memberikan hasil terbaik karena mampu menangkap fitur lokal yang penting untuk klasifikasi gambar. Ukuran filter terlalu besar menyebabkan penurunan performa, kemungkinan karena kehilangan detail penting dan meningkatnya parameter yang sulit dilatih.

4. Pengaruh Jenis *Pooling Layer*

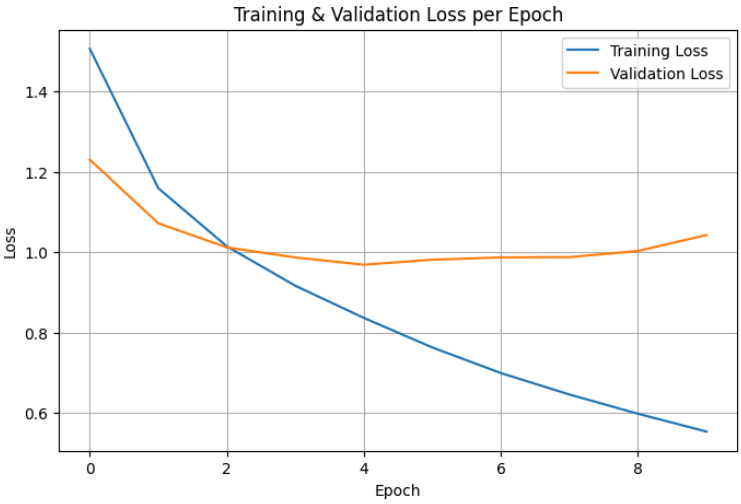
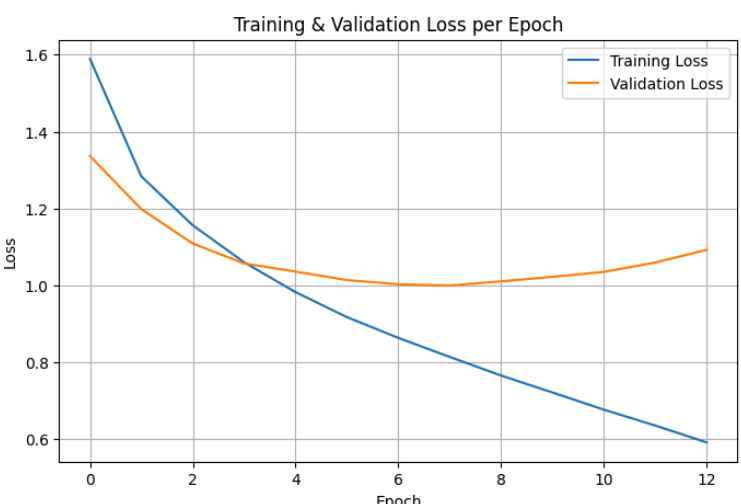
Untuk melihat bagaimana pengaruh jenis *pooling layer* yang digunakan terhadap model, *hyperparameter* berikut dibuat tetap:

- Jumlah *layer* konvolusi: 2 *layer* dengan 32 filter dan 64 filter,
- Ukuran filter per *layer* konvolusi: (3, 3),
- Fungsi aktivasi pada *layer* konvolusi: ReLU,
- *Pooling layer*: ukuran *pool* (2, 2),
- *Batch size*: 64,
- *Epoch*: 30,
- Satu *flatten layer*,
- Satu *dense layer* dengan 128 neuron, fungsi aktivasi ReLU,
- Satu *dense layer* dengan 10 neuron sebagai *output layer*; fungsi aktivasi *softmax*.

a. Hasil Pengujian

Tabel 3.1.4. Hasil Percobaan Pengaruh Jenis *Pooling Layer*

Jenis <i>Pooling Layer</i>	Macro F1-Score	Grafik Training Loss dan Validation Loss
----------------------------	----------------	--

<i>Max Pooling</i>	0.6692	 Gambar 3.1.10. Grafik <i>Loss</i> Model dengan <i>Max Pooling</i>
<i>Average Pooling</i>	0.6628	 Gambar 3.1.11. Grafik <i>Loss</i> Model dengan <i>Average Pooling</i>

b. Pembahasan

- Hasil Akhir Prediksi (*Macro F1-Score*)

Hasil *Macro F1-Score* menunjukkan bahwa *Max Pooling* (0.6692) memberikan kinerja yang sedikit lebih baik dibandingkan dengan *Average Pooling* (0.6628) untuk *dataset* dan konfigurasi model ini. Perbedaannya tidak terlalu besar, namun *Max Pooling* unggul tipis.

- Perbandingan Grafik *Training & Validation Loss*

1. *Max Pooling*

Training loss menurun secara konsisten selama 9 *epoch*. *Validation loss* turun dan mencapai titik terendah sekitar 0.97 pada *epoch* ke-4, kemudian cenderung stagnan dan sedikit meningkat pada *epoch* ke-9. Model dengan *Max Pooling* menunjukkan penurunan *training loss* yang baik. *Overfitting* mulai terlihat setelah *epoch* ke-4, di mana *validation loss* berhenti menurun dan mulai sedikit naik.

2. *Average Pooling*

Training loss menurun secara konsisten selama 12 *epoch*. *Validation loss* turun dan mencapai titik terendah antara *epoch* ke-6 dan ke-7, kemudian mulai meningkat pada *epoch* ke-12. Model dengan *Average Pooling* juga menunjukkan penurunan *training loss* yang baik, meskipun *validation loss* minimumnya sedikit lebih tinggi dan dicapai pada *epoch* yang lebih lambat dibandingkan *Max Pooling*. *Overfitting* mulai terlihat lebih jelas setelah *epoch* ke-7. Celah antara *training* dan *validation loss* di akhir tampak sedikit lebih lebar.

- Kesimpulan Perbandingan Grafik *Training & Validation Loss*,

Dari grafik *loss*, kedua jenis *pooling* menunjukkan pola *overfitting* yang serupa. Namun, *Max Pooling* mencapai *validation loss* minimum yang sedikit lebih rendah dan pada *epoch* yang lebih awal, yang bisa mengindikasikan pembelajaran yang sedikit lebih efisien atau generalisasi yang sedikit lebih baik sebelum *overfitting*.

3.2. Simple Recurrent Neural Network (Simple RNN)

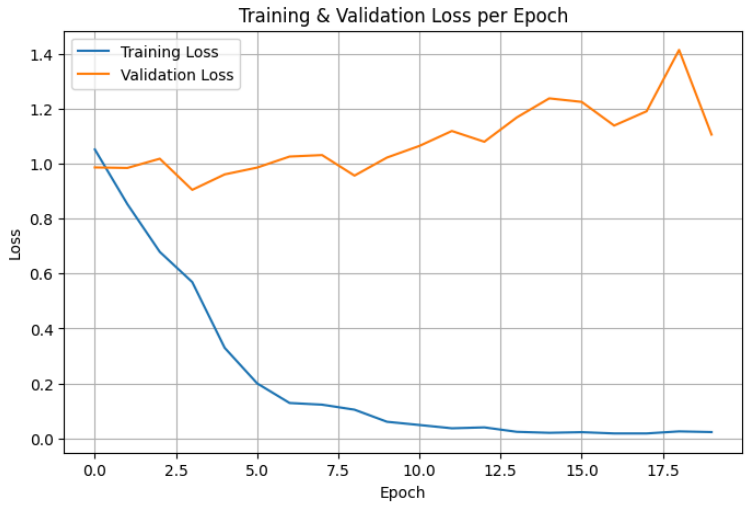
1. Pengaruh Jumlah Layer RNN

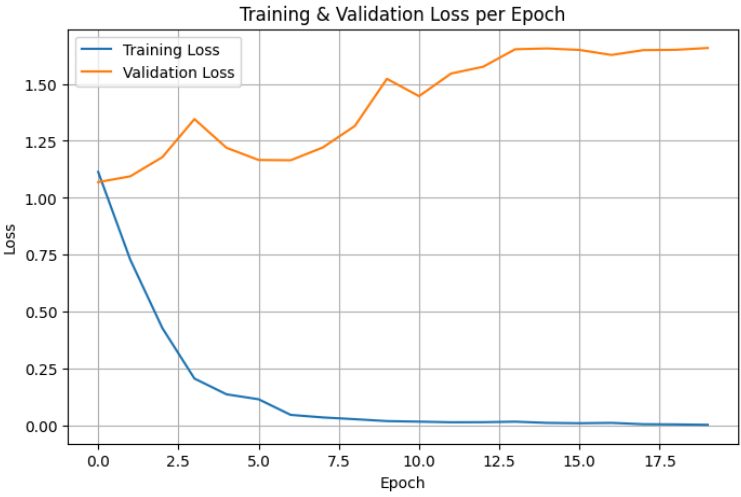
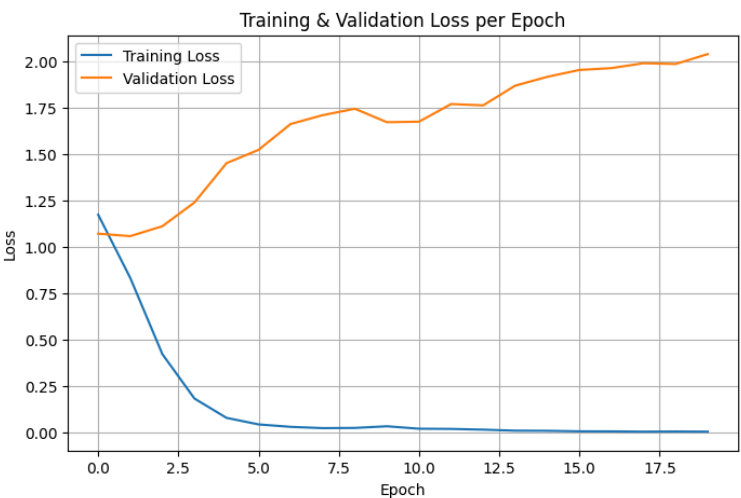
Untuk melihat bagaimana pengaruh jumlah *layer* RNN terhadap model, *hyperparameter* berikut dibuat tetap:

- Jumlah *cell* tiap *layer* RNN: 32,
- Fungsi aktivasi pada *layer* RNN: *tanh*,
- Vektor *embedding* berukuran 128,
- Arah: *bidirectional*,
- *Batch size*: 32,
- *Epoch*: 20,
- Satu *dropout layer* dengan *rate* 0.5.
- Satu *dense layer* dengan 3 neuron sebagai *output layer*; fungsi aktivasi *softmax*.

a. Hasil Pengujian

Tabel 3.2.1. Hasil Percobaan Pengaruh Jumlah Layer RNN

Jumlah Layer RNN	Macro F1-Score	Grafik Training Loss dan Validation Loss																														
1 Layer	0.5738	 The graph displays the training and validation loss over 18 epochs. The training loss (blue line) starts at approximately 1.05 and decreases steadily to about 0.05 by epoch 18. The validation loss (orange line) starts at approximately 1.0, fluctuates between 0.9 and 1.4, and ends at approximately 1.1. The legend indicates 'Training Loss' in blue and 'Validation Loss' in orange. <table border="1"><caption>Approximate data points from Gambar 3.2.1</caption><thead><tr><th>Epoch</th><th>Training Loss</th><th>Validation Loss</th></tr></thead><tbody><tr><td>0.0</td><td>1.05</td><td>1.00</td></tr><tr><td>2.5</td><td>0.65</td><td>1.02</td></tr><tr><td>5.0</td><td>0.25</td><td>0.98</td></tr><tr><td>7.5</td><td>0.15</td><td>1.05</td></tr><tr><td>10.0</td><td>0.10</td><td>1.10</td></tr><tr><td>12.5</td><td>0.08</td><td>1.25</td></tr><tr><td>15.0</td><td>0.07</td><td>1.20</td></tr><tr><td>17.5</td><td>0.06</td><td>1.40</td></tr><tr><td>18.0</td><td>0.05</td><td>1.10</td></tr></tbody></table> Gambar 3.2.1. Grafik <i>Loss</i> Model dengan 1 Layer Simple RNN	Epoch	Training Loss	Validation Loss	0.0	1.05	1.00	2.5	0.65	1.02	5.0	0.25	0.98	7.5	0.15	1.05	10.0	0.10	1.10	12.5	0.08	1.25	15.0	0.07	1.20	17.5	0.06	1.40	18.0	0.05	1.10
Epoch	Training Loss	Validation Loss																														
0.0	1.05	1.00																														
2.5	0.65	1.02																														
5.0	0.25	0.98																														
7.5	0.15	1.05																														
10.0	0.10	1.10																														
12.5	0.08	1.25																														
15.0	0.07	1.20																														
17.5	0.06	1.40																														
18.0	0.05	1.10																														

2 Layer	0.4806	 Gambar 3.2.2. Grafik <i>Loss</i> Model dengan 2 Layer Simple RNN
3 Layer	0.4149	 Gambar 3.2.3. Grafik <i>Loss</i> Model dengan 3 Layer Simple RNN

b. Pembahasan

- Hasil Akhir Prediksi (*Macro F1-Score*)

Berdasarkan hasil pengujian, penambahan jumlah layer RNN justru menunjukkan penurunan *macro F1-score* secara konsisten (1 Layer: 0.5738, 2 Layer: 0.4806, 3 Layer: 0.4149). Hal ini mengindikasikan bahwa untuk Simple RNN, model yang lebih dalam (banyak *layer*) cenderung lebih sulit untuk dilatih dan rentan terhadap masalah seperti *vanishing/exploding gradients*, serta *overfitting* karena peningkatan kompleksitas tanpa mekanisme penanganan memori jangka panjang yang efektif seperti pada LSTM.

- Perbandingan Grafik *Training & Validation Loss*

1. 1 *Layer*

Training Loss menurun, meskipun mungkin tidak terlalu tajam. *Validation Loss* cenderung fluktuatif dan mulai menunjukkan divergensi (naik) relatif cepat, mengindikasikan *overfitting* yang cepat terjadi.

2. 2 *Layer*

Training Loss turun lebih rendah, tetapi *Validation Loss* lebih tinggi dan menunjukkan *overfitting* yang lebih parah dan lebih awal dibandingkan model 1 layer. Model menjadi lebih kompleks dan sulit menggeneralisasi.

3. 3 *Layer*

Pola *overfitting* pada *Validation Loss* menjadi semakin jelas dan parah. *Training Loss* bisa sangat rendah, tetapi perbedaan dengan *Validation Loss* sangat signifikan, menunjukkan kegagalan generalisasi yang paling buruk di antara ketiganya.

- Kesimpulan Perbandingan Grafik *Training & Validation Loss*

Menambah jumlah *layer* pada *Simple RNN* secara umum memperburuk kemampuan generalisasi model. Model menjadi lebih cepat *overfitting* dan *macro F1-score* menurun. Ini menunjukkan bahwa arsitektur *Simple RNN* yang lebih dalam tidak efektif untuk *dataset* dan konfigurasi yang diuji, kemungkinan karena kesulitan dalam menangkap dependensi jangka panjang.

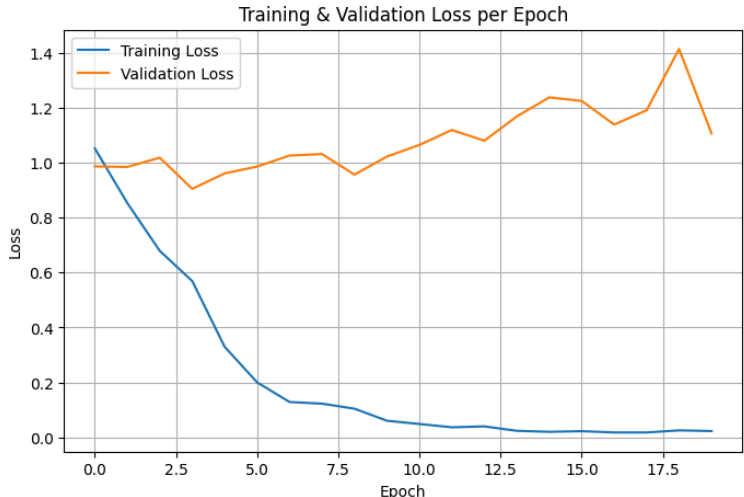
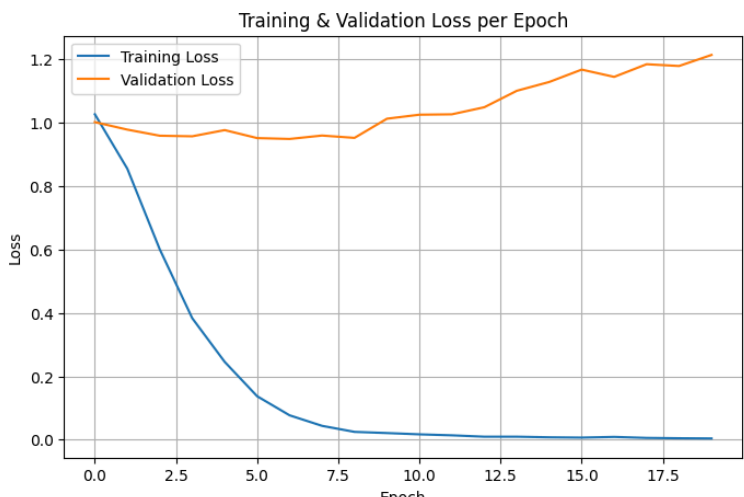
2. Pengaruh Banyak *Cell RNN* per *Layer*

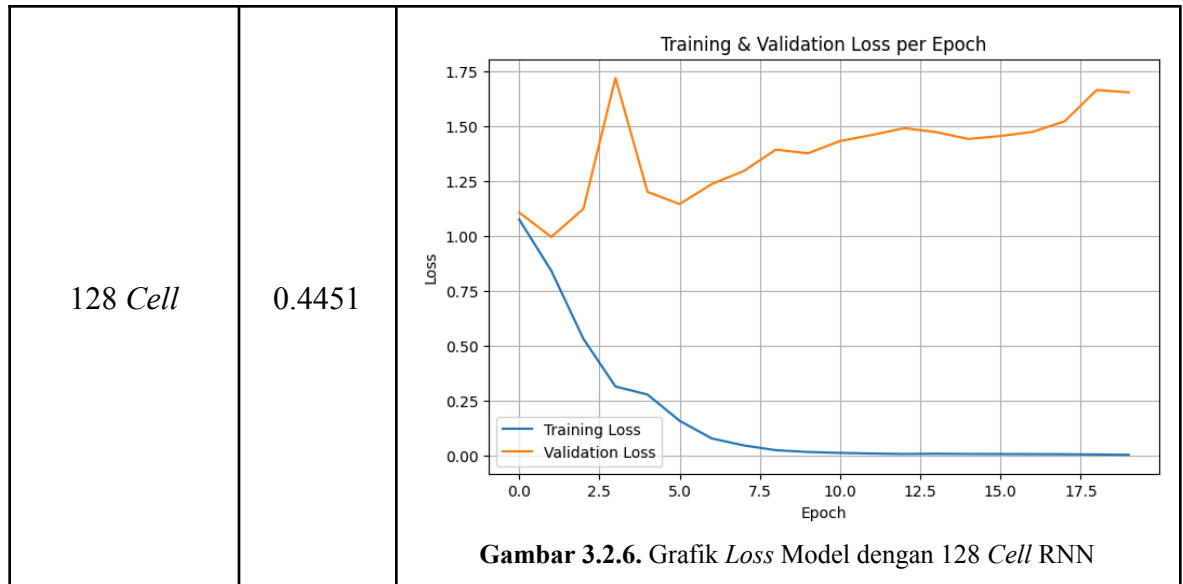
Untuk melihat bagaimana pengaruh banyaknya *cell* per *layer RNN* terhadap model, *hyperparameter* berikut dibuat tetap:

- Jumlah *layer RNN*: 1,
- Fungsi aktivasi pada *layer RNN*: tanh,
- Vektor *embedding* berukuran 128,
- Arah: *bidirectional*,
- *Batch size*: 32,
- *Epoch*: 20,
- Satu *dropout layer* dengan *rate* 0.5.
- Satu *dense layer* dengan 3 neuron sebagai *output layer*, fungsi aktivasi *softmax*.

a. Hasil Pengujian

Tabel 3.2.2. Hasil Percobaan Pengaruh Banyak *Cell* RNN per *Layer*

Jumlah <i>Cell</i> per <i>Layer</i> RNN	Macro F1-Score	Grafik Training Loss dan Validation Loss
32 <i>Cell</i>	0.5738	 Gambar 3.2.4. Grafik <i>Loss</i> Model dengan 32 <i>Cell</i> RNN
64 <i>Cell</i>	0.6181	 Gambar 3.2.5. Grafik <i>Loss</i> Model dengan 64 <i>Cell</i> RNN



b. Pembahasan

- Hasil Akhir Prediksi (*Macro F1-Score*)

Hasil pengujian menunjukkan bahwa peningkatan jumlah *cell* dari 32 (*F1-Score*: 0.5738) ke 64 (*F1-Score*: 0.6181) memberikan peningkatan performa. Namun, penambahan lebih lanjut ke 128 *cell* (*F1-Score*: 0.4451) justru menurunkan *F1-Score* secara drastis. Ini menyiratkan bahwa ada kapasitas optimal; terlalu sedikit *cell* mungkin menyebabkan *underfitting*, sementara terlalu banyak *cell* dapat menyebabkan *overfitting* atau model menjadi terlalu kompleks untuk dilatih dengan baik.

- Perbandingan Grafik *Training & Validation Loss*

1. 32 Cell per Layer

Training Loss menurun, dan *Validation Loss* menunjukkan pola *overfitting* setelah beberapa *epoch*.

2. 64 Cell per Layer

Training Loss menurun sedikit lebih baik. *Validation Loss* menunjukkan performa terbaik di antara ketiganya, dengan *overfitting* yang muncul lebih lambat atau tidak separah konfigurasi lain, selaras dengan *F1-Score* tertinggi.

3. 128 Cell per Layer

Training Loss kemungkinan besar yang terendah, tetapi *Validation Loss* sangat tinggi dan menunjukkan *overfitting* yang signifikan dan cepat. Ini mengindikasikan bahwa kapasitas model terlalu besar untuk data yang ada.

- Kesimpulan Perbandingan Grafik *Training & Validation Loss*

Jumlah *cell* yang optimal (dalam kasus ini 64 *cell*) memberikan keseimbangan antara kapasitas model untuk belajar dan kemampuan generalisasi. Menambah *cell* secara berlebihan pada Simple RNN dapat dengan cepat menyebabkan *overfitting* dan penurunan performa, yang tercermin pada *Macro F1-Score* dan perilaku *Validation Loss*.

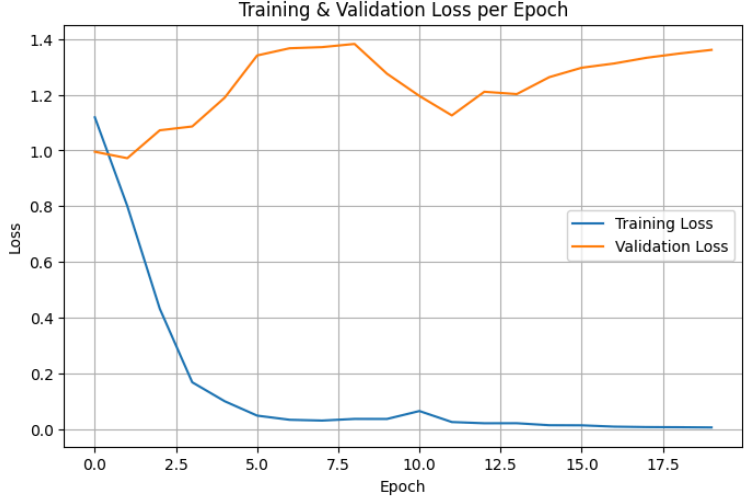
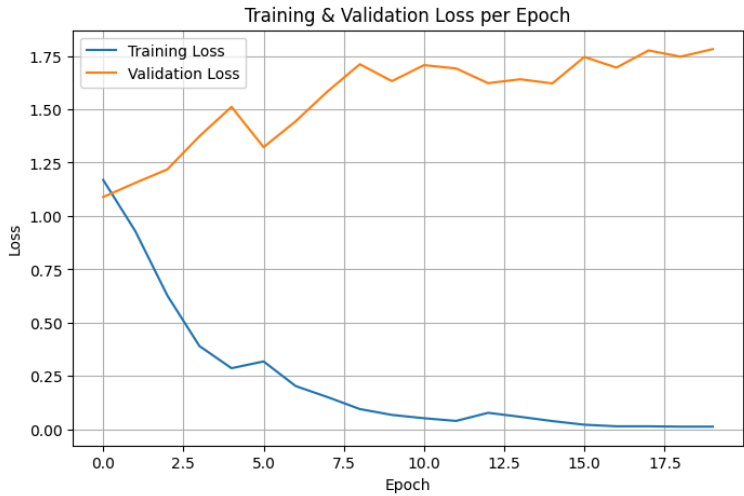
3. Pengaruh Jenis Layer RNN Berdasarkan Arah

Untuk melihat bagaimana pengaruh jenis arah *forward* yang digunakan terhadap model, *hyperparameter* berikut dibuat tetap:

- Jumlah *layer* RNN: 2,
- Jumlah *cell* tiap *layer* RNN: 32,
- Fungsi aktivasi pada *layer* RNN: tanh,
- Vektor *embedding* berukuran 128,
- *Batch size*: 32,
- *Epoch*: 20,
- Satu *dropout layer* dengan *rate* 0.5.
- Satu *dense layer* dengan 3 neuron sebagai *output layer*; fungsi aktivasi *softmax*.

a. Hasil Pengujian

Tabel 3.2.3. Hasil Percobaan Pengaruh Jenis *Layer* RNN Berdasarkan Arah

Arah <i>Layer</i> RNN	Macro F1-Score	Grafik Training Loss dan Validation Loss
<i>Bidirectional</i>	0.5031	 Gambar 3.2.7. Grafik <i>Loss</i> Model dengan <i>Bidirectional</i> RNN
<i>Unidirectional</i>	0.3867	 Gambar 3.2.8. Grafik <i>Loss</i> Model dengan <i>Unidirectional</i> RNN

b. Pembahasan

- Hasil Akhir Prediksi (*Macro F1-Score*)

Penggunaan *layer bidirectional* (*F1-Score*: 0.5031) memberikan peningkatan performa yang signifikan dibandingkan dengan *layer unidirectional* (*F1-Score*: 0.3867). Hal ini terjadi karena *layer bidirectional* memproses sekuens

dari dua arah (maju dan mundur), sehingga dapat menangkap konteks masa lalu dan masa depan pada setiap *timestep*.

- Perbandingan Grafik *Training & Validation Loss*

1. *Bidirectional*

Training Loss dan *Validation Loss* cenderung lebih baik. *Validation Loss* mungkin lebih stabil dan menunjukkan *overfitting* yang lebih lambat karena model memiliki pemahaman konteks yang lebih kaya.

2. *Unidirectional*

Training Loss tetap menurun, tetapi *Validation Loss* cenderung lebih tinggi dan lebih cepat menunjukkan *overfitting* dibandingkan model *bidirectional*, karena hanya mempertimbangkan konteks masa lalu.

- Kesimpulan Perbandingan Grafik *Training & Validation Loss*

Layer Bidirectional secara umum meningkatkan kemampuan model *Simple RNN* dalam memahami konteks sekuensial, yang berdampak positif pada *F1-Score* dan stabilitas *loss*. Model menjadi lebih baik dalam generalisasi karena dapat memanfaatkan informasi dari kedua arah sekuens.

3.3. Long Short-Term Memory (LSTM)

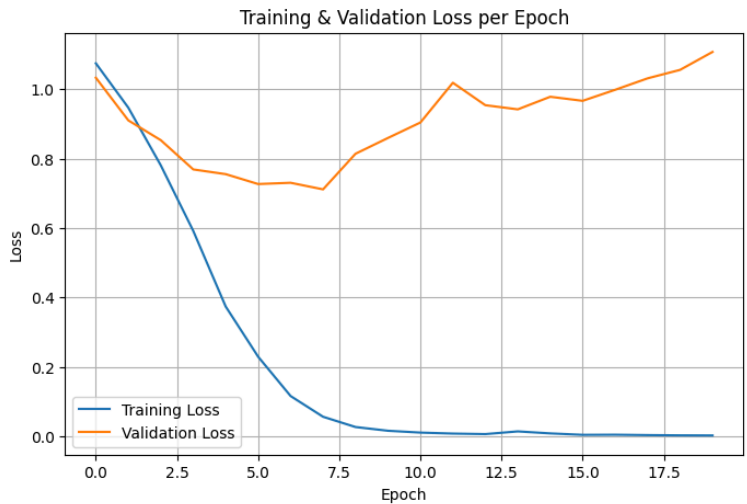
1. Pengaruh Jumlah Layer LSTM

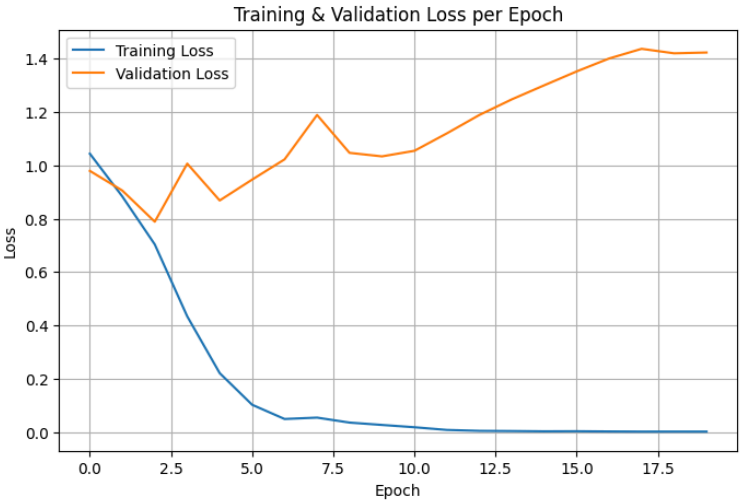
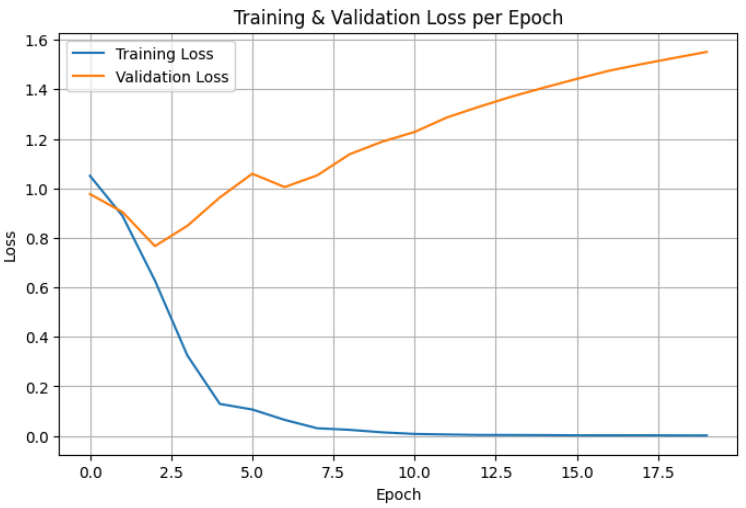
Untuk melihat bagaimana pengaruh jumlah *layer* LSTM terhadap model, *hyperparameter* berikut dibuat tetap:

- Jumlah *cell* tiap *layer* LSTM: 64,
- Vektor *embedding* berukuran 128,
- Arah: *bidirectional*,
- *Batch size*: 32,
- *Epoch*: 20,
- Satu *dropout layer* dengan *rate* 0.5.
- Satu *dense layer* dengan 3 neuron sebagai *output layer*, fungsi aktivasi *softmax*.

a. Hasil Pengujian

Tabel 3.3.1. Hasil Percobaan Pengaruh Jumlah *Layer* LSTM

Jumlah Layer LSTM	Macro F1-Score	Grafik Training Loss dan Validation Loss
1 Layer	0.7052	 Gambar 3.3.1. Grafik <i>Loss</i> Model dengan 1 <i>Layer</i> LSTM

2 Layer	0.7609	 Gambar 3.3.2. Grafik <i>Loss</i> Model dengan 2 Layer LSTM
3 Layer	0.7467	 Gambar 3.3.3. Grafik <i>Loss</i> Model dengan 3 Layer LSTM

b. Pembahasan

- Hasil Akhir Prediksi (*Macro F1-Score*)

Penambahan jumlah layer LSTM dari 1 (*F1-Score*: 0.7052) ke 2 (*F1-Score*: 0.7609) menunjukkan peningkatan performa. Namun, penambahan lebih lanjut ke 3 layer (*F1-Score*: 0.7467) sedikit menurunkan performa. Ini mengindikasikan bahwa arsitektur LSTM yang lebih dalam dapat menangkap dependensi yang lebih kompleks hingga titik tertentu, setelah itu penambahan layer mungkin menyebabkan *overfitting* atau kesulitan optimasi.

- Perbandingan Grafik *Training & Validation Loss*

1. 1 *Layer*

Training Loss menurun. *Validation Loss* juga menurun di awal, kemudian cenderung naik, menunjukkan potensi *overfitting* atau model belum cukup kompleks.

2. 2 *Layer*

Training Loss menurun dengan baik. *Validation Loss* menunjukkan hasil yang fluktuatif sebelum tanda-tanda *overfitting* muncul.

3. 3 *Layer*

Training Loss kemungkinan besar yang terendah. Namun, *Validation Loss* menunjukkan *overfitting* yang lebih jelas dan lebih parah dibandingkan model 2 *layer*, atau kesulitan dalam konvergensi yang optimal.

- Kesimpulan Perbandingan Grafik *Training & Validation Loss*

Menambah jumlah *layer* LSTM dari satu ke dua meningkatkan kemampuan model dalam mengekstrak fitur temporal yang kompleks, yang berdampak positif pada *F1-score*. Namun, penambahan *layer* lebih lanjut (tiga *layer*) tidak memberikan peningkatan dan mulai menunjukkan tanda *overfitting* atau *diminishing returns*.

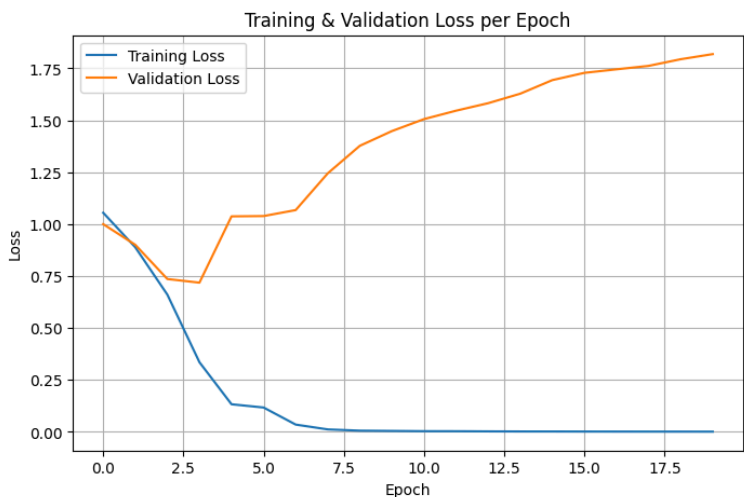
2. Pengaruh Banyak *Cell* LSTM per *Layer*

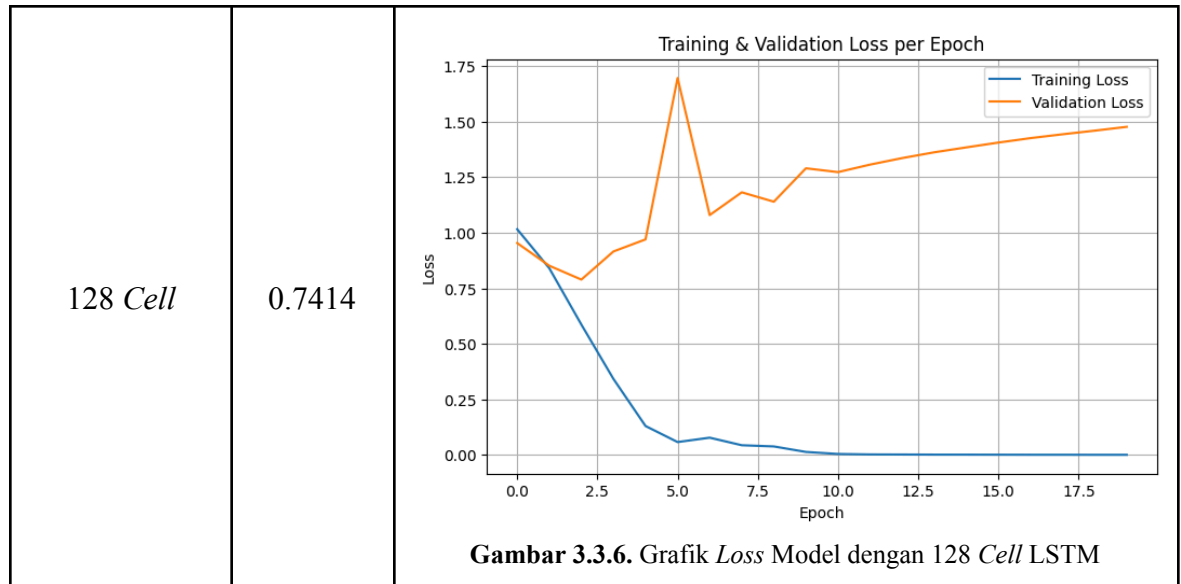
Untuk melihat bagaimana pengaruh banyaknya *cell* per *layer* LSTM terhadap model, *hyperparameter* berikut dibuat tetap:

- Jumlah *layer* LSTM: 2,
- Vektor *embedding* berukuran 128,
- Arah: *bidirectional*,
- *Batch size*: 32,
- *Epoch*: 20,
- Satu *dropout layer* dengan *rate* 0.5.
- Satu *dense layer* dengan 3 neuron sebagai *output layer*; fungsi aktivasi *softmax*.

a. Hasil Pengujian

Tabel 3.3.2. Hasil Percobaan Pengaruh Banyak *Cell* LSTM per *Layer*

Jumlah <i>Cell</i> per <i>Layer</i> LSTM	Macro F1-Score	Grafik Training Loss dan Validation Loss
32 <i>Cell</i>	0.7718	 Gambar 3.3.4. Grafik <i>Loss</i> Model dengan 32 <i>Cell</i> LSTM
64 <i>Cell</i>	0.6955	 Gambar 3.3.5. Grafik <i>Loss</i> Model dengan 64<i>Cell</i> LSTM



b. Pembahasan

- Hasil Akhir Prediksi (*Macro F1-Score*)

Hasil pengujian menunjukkan bahwa 32 cell per layer (*F1-Score*: 0.7718) memberikan performa terbaik. Peningkatan jumlah cell ke 64 (*F1-Score*: 0.6955) justru menurunkan performa, dan peningkatan lebih lanjut ke 128 cell (*F1-Score*: 0.7414) memberikan hasil yang lebih baik dari 64 cell tetapi tidak sebaik 32 cell. Hal ini menunjukkan bahwa untuk konfigurasi 2 layer LSTM pada dataset ini, 32 cell sudah cukup optimal. Jumlah cell yang lebih besar tidak selalu berarti lebih baik dan bisa menyebabkan *overfitting* atau interaksi kompleks yang tidak optimal.

- Perbandingan Grafik *Training & Validation Loss*

1. 32 Cell per Layer

Training Loss menurun. *Validation Loss* menunjukkan perilaku terbaik, dengan penurunan yang baik dan *overfitting* yang relatif terkendali, selaras dengan *F1-Score* tertinggi.

2. 64 Cell per Layer

Training Loss menurun dengan baik, tetapi *Validation Loss* lebih tinggi atau lebih cepat naik (menunjukkan *overfitting*) dibandingkan 32 cell.

3. 128 Cell per Layer

Training Loss kemungkinan besar paling rendah. *Validation Loss* lebih baik dari 64 *cell* tetapi menunjukkan *overfitting* yang lebih jelas dibandingkan 32 *cell*, mengindikasikan kapasitas model yang terlalu besar atau kesulitan optimasi.

- Kesimpulan Perbandingan Grafik *Training & Validation Loss*

Jumlah *cell* yang lebih sedikit (32 *cell*) dalam konfigurasi 2 layer LSTM ini memberikan keseimbangan terbaik antara kapasitas model dan generalisasi. Menambah jumlah *cell* tidak selalu meningkatkan performa dan dapat menyebabkan *overfitting*, yang terlihat dari divergensi antara *Training Loss* dan *Validation Loss*.

3. Pengaruh Jenis Layer LSTM Berdasarkan Arah

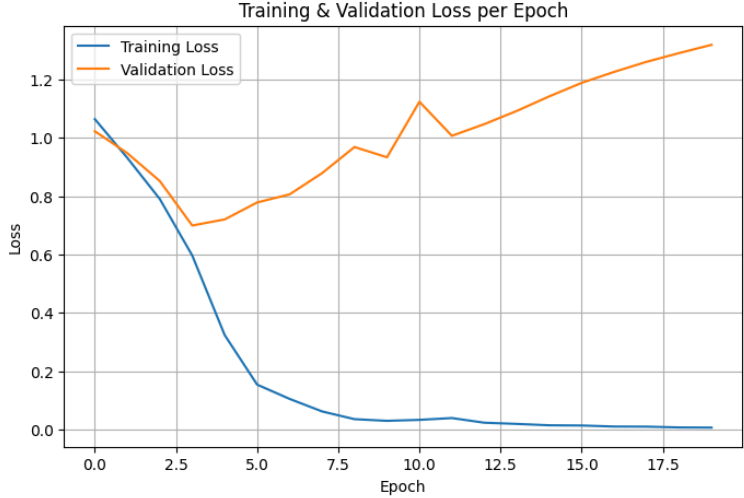
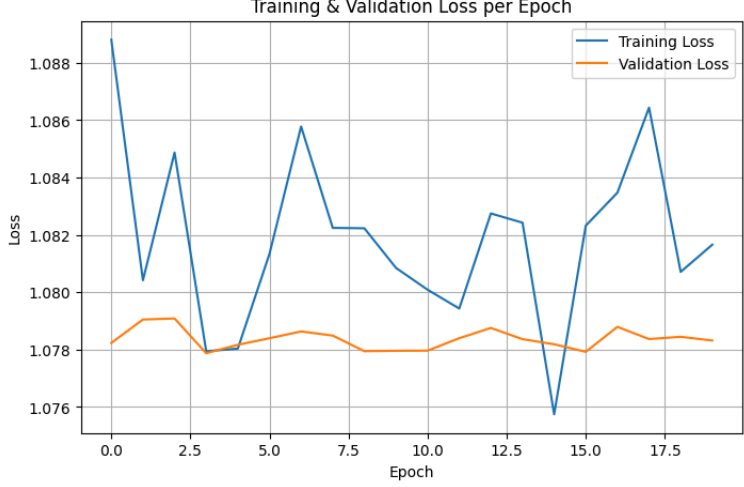
Untuk melihat bagaimana pengaruh jenis arah *forward* yang digunakan terhadap model, *hyperparameter* berikut dibuat tetap:

- Jumlah *layer* LSTM: 2,
- Jumlah *cell* tiap *layer* LSTM: 32,
- Vektor *embedding* berukuran 128,
- *Batch size*: 32,
- *Epoch*: 20,
- Satu *dropout layer* dengan *rate* 0.5.
- Satu *dense layer* dengan 3 neuron sebagai *output layer*; fungsi aktivasi *softmax*.

a. Hasil Pengujian

Tabel 3.3.3. Hasil Percobaan Pengaruh Jenis Layer LSTM Berdasarkan Arah

Arah Layer LSTM	Macro F1-Score	Grafik Training Loss dan Validation Loss
-----------------	----------------	--

<i>Bidirectional</i>	0.7718	 Gambar 3.3.7. Grafik <i>Loss</i> Model dengan <i>Bidirectional</i> LSTM
<i>Unidirectional</i>	0.1844	 Gambar 3.3.8. Grafik <i>Loss</i> Model dengan <i>Unidirectional</i> LSTM

b. Pembahasan

- Hasil Akhir Prediksi (*Macro F1-Score*)

Penggunaan layer *bidirectional* LSTM (*F1-Score*: 0.7718) menunjukkan performa yang jauh lebih unggul dibandingkan dengan layer *unidirectional* LSTM (*F1-Score*: 0.1844). Kemampuan LSTM untuk memproses sekuens dari kedua arah memberikan pemahaman konteks yang jauh lebih kaya, yang sangat krusial untuk tugas-tugas sekuensial.

- Perbandingan Grafik *Training & Validation Loss*

1. *Bidirectional*

Training Loss dan *Validation Loss* menunjukkan performa yang jauh lebih baik. *Validation Loss* cenderung lebih rendah, lebih stabil, dan *overfitting* muncul lebih lambat dan lebih minim, mencerminkan pemahaman model yang sangat baik.

2. *Unidirectional*

Training Loss sangat fluktuatif dan cukup tinggi. *Validation Loss* cukup tinggi tetapi menunjukkan *trend* yang konstan, menunjukkan performa generalisasi yang buruk dan *overfitting*. Model kesulitan menangkap pola yang signifikan hanya dengan informasi satu arah.

- Kesimpulan Perbandingan Grafik *Training & Validation Loss*

Layer bidirectional secara signifikan meningkatkan kemampuan model LSTM dalam memproses dan memahami informasi sekuensial dengan mempertimbangkan konteks masa lalu dan masa depan. Hal ini berdampak besar pada peningkatan *Macro F1-Score* dan stabilitas serta rendahnya *Validation Loss*.

3.4. Perbandingan Hasil *Forward Propagation from Scratch* dengan *Library*

1. CNN

a. Hasil Pengujian

Tabel 3.4.1. Perbandingan Hasil *Forward Propagation from Scratch* dengan *Library* untuk CNN

Konfigurasi	<i>Macro F1-Score:</i> Keras	<i>Macro F1-Score:</i> <i>Scratch</i>
Conv2D(8, (3, 3), activation='relu'), MaxPooling2D((2, 2)), Flatten(), Dense(128, activation='relu'), Dense(10, activation='softmax')	0.5824	0.5824
Conv2D(2, (3, 3), activation='relu'), MaxPooling2D((2, 2)), Conv2D(2, (3, 3), activation='relu'), AveragePooling2D((2, 2)), GlobalAveragePooling2D(), Dense(128, activation='relu'), Dense(10, activation='softmax')	0.2492	0.2492

b. Pembahasan

Hasil *Macro F1-Score* yang diperoleh dari implementasi *from scratch* (0.5824 dan 0.2492) identik dengan hasil yang diperoleh menggunakan library Keras untuk kedua konfigurasi CNN yang diuji. Kesamaan hasil ini menunjukkan bahwa implementasi manual dari proses *forward propagation* untuk layer-layer CNN, telah dilakukan dengan benar dan menghasilkan output yang setara secara numerik dengan implementasi *library*. Hal ini memvalidasi akurasi perhitungan dalam modul *from scratch*.

2. Simple RNN

a. Hasil Pengujian

Tabel 3.4.2. Perbandingan Hasil *Forward Propagation from Scratch* dengan *Library* untuk Simple RNN

Konfigurasi	Macro F1-Score: Keras	Macro F1-Score: Scratch
Input(shape=(100,)), Embedding(input_dim=10000, output_dim=128), Bidirectional(SimpleRNN(64, return_sequences=True)), Bidirectional(SimpleRNN(32)), Dropout(0.5), Dense(3, activation='softmax')	0.4456	0.4456
Input(shape=(100,)), Embedding(input_dim=10000, output_dim=128), SimpleRNN(64, return_sequences=True), SimpleRNN(32), Dropout(0.5), Dense(3, activation='softmax')	0.3217	0.3217

b. Pembahasan

Untuk kedua konfigurasi Simple RNN yang diuji, Macro F1-Score dari implementasi *from scratch* (0.4456 dan 0.3217) juga menunjukkan kesamaan yang persis dengan hasil dari *library* Keras. Hal ini mengkonfirmasi bahwa implementasi *forward propagation* SimpleRNN pada modul *from scratch* sudah akurat. Kemampuan untuk mereplikasi hasil *library* menunjukkan bahwa logika dan perhitungan matematis untuk setiap langkah dalam *forward propagation* RNN telah diimplementasikan dengan tepat.

3. LSTM

a. Hasil Pengujian

Tabel 3.4.3. Perbandingan Hasil *Forward Propagation from Scratch* dengan *Library* untuk LSTM

Konfigurasi	<i>Macro F1-Score:</i> Keras	<i>Macro F1-Score:</i> <i>Scratch</i>
Input(shape=(100,)), Embedding(input_dim=10000, output_dim=128), Bidirectional(LSTM(64, return_sequences=True)), Bidirectional(LSTM(32)), Dropout(0.5), Dense(3, activation='softmax')	0.7514	0.7514
Input(shape=(100,)), Embedding(input_dim=10000, output_dim=128), LSTM(64, return_sequences=True), LSTM(32), Dropout(0.5), Dense(3, activation='softmax')	0.1844	0.1844

b. Pembahasan

Serupa dengan CNN dan *Simple RNN*, hasil *Macro F1-Score* untuk model LSTM yang diimplementasikan *from scratch* (0.7514 dan 0.1844) sama persis dengan yang dihasilkan oleh *library* Keras pada kedua konfigurasi. Kesamaan ini memvalidasi kebenaran implementasi *forward propagation* untuk layer LSTM dalam modul *from scratch*. Ini menunjukkan bahwa komputasi internal LSTM yang kompleks telah berhasil direplikasi dengan akurat.

BAB IV

KESIMPULAN DAN SARAN

4.1. Kesimpulan

Implementasi modul *forward propagation from scratch* untuk arsitektur *Convolutional Neural Network* (CNN), *Simple Recurrent Neural Network* (Simple RNN), dan *Long-Short Term Memory* (LSTM) berhasil dilakukan dengan pustaka komputasi numerik dasar NumPy. Validasi terhadap implementasi *library* Keras menunjukkan bahwa *modul from scratch* mampu menghasilkan prediksi dengan nilai *Macro F1-Score* yang identik, mengonfirmasi akurasi perhitungan matematis dan logika yang diimplementasikan untuk setiap *layer*. Analisis pengaruh *hyperparameter* menunjukkan bahwa konfigurasi arsitektur sangat krusial. Pada CNN, penambahan *layer* konvolusi hingga 3 *layer*, jumlah *filter* yang optimal (misalnya [32, 64, 128]), ukuran *filter* kecil (3x3), dan penggunaan *Max Pooling* memberikan hasil terbaik. Untuk Simple RNN, model yang lebih dangkal (1 *layer*) dengan jumlah *cell* optimal (64 *cell*) dan arsitektur *bidirectional* lebih unggul, karena penambahan kedalaman dengan cepat menurunkan performa. LSTM menunjukkan kemampuan menangani kedalaman yang lebih baik (optimal pada 2 *layer* dengan 32 *cell*) dan juga sangat diuntungkan oleh arsitektur *bidirectional*, menghasilkan performa tertinggi untuk *sequential task*.

4.2. Saran

Untuk pengembangan dan analisis model yang lebih komprehensif, beberapa aspek dapat dipertimbangkan. Pertama, melakukan eksplorasi *hyperparameter* yang lebih luas dan sistematis, seperti variasi jumlah *unit/filter* per *layer*, ukuran *kernel*, jenis fungsi aktivasi, dan parameter *regularization* (misalnya, *dropout rate*) untuk menemukan konfigurasi optimal pada setiap arsitektur dan *dataset* yang digunakan. Kedua, penerapan teknik augmentasi data yang lebih beragam untuk *dataset* citra (CNN) dan teknik pra-pemrosesan teks yang lebih canggih untuk *dataset* sekuensial (RNN/LSTM), seperti penggunaan *pre-trained word embeddings* yang berbeda atau normalisasi teks yang lebih detail, dapat meningkatkan kualitas input dan performa model.

PEMBAGIAN TUGAS

NIM	Nama	Pembagian Kerja
13522006	Agil Fadillah Sabri	Implementasi kelas Layer (Conv2D, Pooling, Flatten, GlobalPooling, Bidirectional, LSTM, Dense) dan Model; pengujian <i>hyperparameter</i> ; pembuatan laporan.
13522034	Bastian H. Suryapratama	Implementasi kelas Layer (SimpleRNN); pengujian <i>hyperparameter</i> ; pembuatan laporan.
13522052	Haikal Assyauqi	Implementasi kelas Layer (Embedding); pengujian <i>hyperparameter</i> ; pembuatan laporan.

REFERENSI

Keras. (n.d.). *Keras 3 API documentation*. Keras. <https://keras.io/api/>.

Tim Pengajar IF3270. (2025). *Convolutional Neural Network*.
https://edunexcontentprodhot.blob.core.windows.net/edunex/nullfile/1743979407057_IF3270-Mgg0506-CNN?sv=2024-11-04&spr=https&st=2025-04-06T21%3A43%3A16Z&se=2027-04-06T21%3A43%3A16Z&sr=b&sp=r&sig=%2FBujVcwCfyRDlqz7YE0uomZ7LlaHZpmXrap4pe6g85s%3D&rsct=application%2Fpdf.

Tim Pengajar IF3270. (2025). *Recurrent Neural Network (RNN)*.
https://edunexcontentprodhot.blob.core.windows.net/edunex/nullfile/1745821108485_IF3270-Mgg09-RNN-part1?sv=2024-11-04&spr=https&st=2025-04-28T06%3A18%3A28Z&se=2027-04-28T06%3A18%3A28Z&sr=b&sp=r&sig=BmVDnt5AoFWMuz9dIBOzKx5IA1AHvSLfJdamQTpih8I%3D&rsct=application%2Fpdf.

Tim Pengajar IF3270. (2025). *Recurrent Neural Network (ANN)*.
https://edunexcontentprodhot.blob.core.windows.net/edunex/nullfile/1747619176390_IF3270-Mgg11-RNN-part2?sv=2024-11-04&spr=https&st=2025-05-19T01%3A46%3A11Z&se=2027-05-19T01%3A46%3A11Z&sr=b&sp=r&sig=BHBXDJL2Wgw%2BfNwrkW4dqqxmrDVauFWgdng3Jf151A%3D&rsct=application%2Fpdf.